SILVER OAK UNIVERSITY
EDUCATION TO INNOVATION

# MINOR PROJECT REPORT

*Entitled*

# AI-BASED MOCK INTERVIEW PLATFORM

*Submitted by*

**Dhruv K. Parmar — (2401031830002)**

**Pratiksinh K. Chauhan — (2301031800123)**

**Dhwij Harishkumar Patel — (2301031800147)**

*in*

**PARTIAL FULFILLMENT FOR THE DEGREE OF**

**BACHELOR OF TECHNOLOGY**

*in*

**COMPUTER SCIENCE AND ENGINEERING**

**Department of Computer Engineering**

**College of Technology**

**Silver Oak University, Ahmedabad**

**[October, 2026]**

**College of Technology**

**Department of Computer Engineering**

# CERTIFICATE

This is to certify that **Dhruv K. Parmar (2401031830002)**, **Pratiksinh K. Chauhan (2301031800123)** and **Dhwij Harishkumar Patel (2301031800147)** have submitted the Minor Project report entitled **"AI-Based Mock Interview Platform: An LLM-Powered Technical Interview and Skill Assessment System"** in partial fulfillment of the requirement for the award of the degree in Bachelor of Technology in **Computer Science and Engineering** of Silver Oak University, Ahmedabad during the academic year 2026-2027.

**Dr. Monali Suthar**
**Project Guide**
**Department of Computer Engineering**
**College of Technology**
**Silver Oak University**

**Dr. Vikas Tulshyan**
**Head of Department**
**Department of Computer Engineering**
**College of Technology**
**Silver Oak University**

# DECLARATION

We hereby declare that the Minor Project report submitted along with the project entitled **"AI-Based Mock Interview Platform: An LLM-Powered Technical Interview and Skill Assessment System"** submitted in partial fulfillment for the degree of Bachelor of Technology in Computer Science and Engineering to Silver Oak University, Ahmedabad, is a bonafide record of original work carried out by us at the College of Technology under the supervision of Dr. Monali Suthar.

We further declare that the report does not contain any material previously submitted for the award of any degree or diploma in this or any other university. It is also certified that no part of this report has been copied from any source without proper acknowledgment, and all referenced materials, figures and information from other sources have been appropriately cited in the report.

**DHRUV K. PARMAR (2401031830002)** _____

**PRATIKSINH K. CHAUHAN (2301031800123)** _____

**DHWIJ HARISHKUMAR PATEL (2301031800147)** _____

# ACKNOWLEDGEMENT

We would like to express our sincere gratitude to Silver Oak University, Ahmedabad, and the Department of Computer Engineering, College of Technology, for giving us the opportunity to carry out our minor project, “AI-Based Mock Interview Platform”.

We are deeply thankful to our project guide, **Dr. Monali Suthar**, for her continuous guidance, valuable suggestions and encouragement at every stage of the project, from defining the problem to reviewing the implementation and this report.

We also thank **Dr. Vikas Tulshyan**, Head of the Department of Computer Engineering, for providing the facilities and an environment that supported our work. We are grateful to all faculty members of the department for their feedback during project reviews, which helped us improve both the system and its presentation.

Finally, we thank our friends and families for their support, and each member of our team for the cooperation and shared effort that made the design, development, testing and documentation of this project possible.

# ABSTRACT

Technical interviews are a decisive step in campus placements and software hiring, yet most students get very little realistic practice. Human mock interviews are costly and hard to schedule, while coding websites judge only whether code passes tests and ignore how a candidate explains concepts and trade-offs. This project presents the **AI-Based Mock Interview Platform**, a full-stack web application in which a large language model (Google Gemini) acts as a technical interviewer.

A candidate configures a session by target role, experience level, difficulty, interview type, technologies, topics, duration and number of questions. The backend generates one question at a time for the selected topics, evaluates every answer on technical accuracy, depth, problem solving and communication, and can ask one adaptive follow-up question for each main question. Coding questions are answered in an embedded Monaco code editor and reviewed by the model for correctness, algorithm choice, code quality, and time and space complexity. When the interview ends, the system produces an overall score, five skill scores, strengths, weaknesses, recommendations and a written summary. The interview workspace also provides proctoring controls (fullscreen lock, tab-switch detection with three warnings, and blocked clipboard and developer shortcuts), voice answers through the Web Speech API, a practice sandbox, interview history and a performance dashboard.

The system is built with React and TypeScript on the frontend and Node.js with Express on the backend. Every AI response is requested in JSON format and validated with Zod schemas before use, users are authenticated with JSON Web Tokens and bcrypt-hashed passwords, and the data layer supports PostgreSQL through Prisma or an in-memory store for development. In functional testing with the live Gemini model, the platform generated role-specific questions, scored two coding answers at 100/100 and 92/100 with complexity analysis, and produced a final report that correctly reflected an unanswered question. Testing also identified limitations, including dependence on the availability and quota of the external AI service, simulated code execution, and dashboard statistics that are not yet linked to stored results.

**Keywords:** Large Language Models, Mock Interview, Technical Assessment, Gemini API, Automated Code Evaluation, Prompt Engineering, Online Proctoring, React, Node.js.

# LIST OF FIGURES


| | | |
|---|---|---|
| Figure 3.1 | Use Case Diagram of the AI-Based Mock Interview Platform | 10 |
| Figure 3.2 | System Architecture of the AI-Based Mock Interview Platform | 12 |
| Figure 3.3 | Class Diagram of the Backend Service and AI Layers | 14 |
| Figure 3.4 | Entity-Relationship Diagram of the Platform Database | 15 |
| Figure 3.5 | Sequence Diagram of an AI Interview Session | 18 |
| Figure 3.6 | Flowchart of the Interview Process | 19 |
| Figure 4.1 | State Diagram of an Interview Session | 24 |
| Figure 5.1 | Landing Page | 33 |
| Figure 5.2 | Login Page | 33 |
| Figure 5.3 | Sign-up Page | 34 |
| Figure 5.4 | Candidate Dashboard (Sample Data for a New Account) | 34 |
| Figure 5.5 | Interview Setup Page | 35 |
| Figure 5.6 | Proctored Session Start Screen | 35 |
| Figure 5.7 | AI-Generated Coding Question in the Interview Workspace | 36 |
| Figure 5.8 | Code Editor with Execution Terminal | 36 |
| Figure 5.9 | AI Evaluation of a Coding Answer with Complexity Analysis | 37 |
| Figure 5.10 | Text Answer Mode with Question Panel | 37 |
| Figure 5.11 | Voice Answer Mode | 38 |
| Figure 5.12 | Tab-Switch Proctoring Warning | 38 |
| Figure 5.13 | Evaluation Results Overview | 39 |
| Figure 5.14 | Question Review with Answer and Score | 39 |
| Figure 5.15 | Strengths, Weaknesses and Recommendations | 40 |
| Figure 5.16 | Interview Session History (Sample Data for a New Account) | 40 |
| Figure 5.17 | Practice Sandbox | 41 |
| Figure 5.18 | Profile Settings | 41 |
| Figure 5.19 | Settings Panel | 42 |

# LIST OF TABLES


| | | |
|---|---|---|
| Table 2.1 | Summary of Literature Survey | 6 |
| Table 3.1 | Database Entities and Their Purpose | 12 |
| Table 3.2 | REST API Endpoints | 15 |
| Table 4.1 | Technology Stack | 20 |
| Table 4.2 | Interview Configuration Options | 21 |
| Table 4.3 | Answer Evaluation Criteria | 23 |
| Table 4.4 | Mapping of Coding Evaluation to Answer Scores | 24 |
| Table 4.5 | Proctoring Rules in the Interview Workspace | 26 |
| Table 5.1 | Functional Test Cases and Results | 28 |
| Table 5.2 | Observed AI Evaluation in the Sample Session | 30 |

# LIST OF ABBREVIATIONS

| | |
|---|---|
| **AI** | Artificial Intelligence |
| **API** | Application Programming Interface |
| **CORS** | Cross-Origin Resource Sharing |
| **CSS** | Cascading Style Sheets |
| **DB** | Database |
| **ER** | Entity-Relationship |
| **ESM** | ECMAScript Modules |
| **HTML** | HyperText Markup Language |
| **HTTP** | HyperText Transfer Protocol |
| **JSON** | JavaScript Object Notation |
| **JWT** | JSON Web Token |
| **LLM** | Large Language Model |
| **NLP** | Natural Language Processing |
| **ORM** | Object-Relational Mapping |
| **REST** | Representational State Transfer |
| **RLHF** | Reinforcement Learning from Human Feedback |
| **SDK** | Software Development Kit |
| **SDLC** | Software Development Life Cycle |
| **SPA** | Single-Page Application |
| **SQL** | Structured Query Language |
| **T&P** | Training and Placement |
| **UI** | User Interface |
| **UML** | Unified Modeling Language |
| **UUID** | Universally Unique Identifier |
| **UX** | User Experience |

# TABLE OF CONTENTS


| | | |
|---|---|---|
| | Certificate | i |
| | Declaration | ii |
| | Acknowledgement | iii |
| | Abstract | iv |
| | List of Figures | v |
| | List of Tables | vi |
| | List of Abbreviations | vii |
| | Table of Contents | viii |
| **Chapter 1: Introduction** | | **1** |
| 1.1 | Introduction to the Project | 1 |
| 1.2 | Problem Statement | 1 |
| 1.3 | Aim and Objectives | 2 |
| 1.4 | Scope of the Project | 2 |
| 1.5 | Organisation of the Report | 3 |
| **Chapter 2: Literature Review** | | **4** |
| 2.1 | Existing Approaches | 4 |
| 2.2 | Limitations of Existing Approaches | 4 |
| 2.3 | Proposed Approach | 5 |
| 2.4 | Research Papers | 6 |
| **Chapter 3: System Analysis and Design** | | **8** |
| 3.1 | Existing and Proposed System | 8 |
| 3.2 | System Requirements | 8 |
| 3.3 | System Architecture | 11 |
| 3.4 | Database Design | 12 |
| 3.5 | API Design | 15 |
| 3.6 | System Workflow | 16 |
| **Chapter 4: System Implementation** | | **20** |
| 4.1 | Technology Stack | 20 |
| 4.2 | User Authentication and Security | 21 |

4.3 Interview Configuration 21
4.4 AI Question Generation and Prompt Engineering 22
4.5 Answer Evaluation and Adaptive Follow-up 23
4.6 Coding Challenge Evaluation 24
4.7 Result Generation and Score Aggregation 25
4.8 Proctoring and Screen Lock 26
4.9 Voice Input and Practice Sandbox 26
4.10 Dashboard and Interview History 27
4.11 Development Mock AI Mode 27

**Chapter 5: Testing and Results 28**
5.1 Testing Approach 28
5.2 Test Cases and Results 28
5.3 Results and Discussion 30
5.4 System Limitations 31
5.5 System Screenshots 32

**Chapter 6: Conclusion and Future Scope 43**
6.1 Conclusion 43
6.2 Future Scope 43

**References 45**

# CHAPTER 1: INTRODUCTION

## 1.1 INTRODUCTION TO THE PROJECT

Technical interviews are one of the most important stages in software engineering recruitment. Whether in a campus placement drive or an off-campus hiring process, candidates are expected to explain programming concepts, reason about system design, write working code and communicate their thinking clearly, often under time pressure. Many students know the material but have little experience presenting it in an interview setting, and feedback on their performance is rarely available.

The **AI-Based Mock Interview Platform** (shown in the application as "AI Interviewer") is a web application that simulates a technical interview with an AI interviewer. The candidate selects a target role, experience level, difficulty, interview type, technologies and topics. A large language model (LLM), Google Gemini, then asks questions one at a time, evaluates each answer, may ask a follow-up question, and finally produces a detailed evaluation report with an overall score, skill-wise scores, strengths, weaknesses and recommendations.

The platform supports both conceptual and coding questions. Conceptual answers can be typed or spoken through the browser's speech recognition, while coding answers are written in an embedded code editor. To keep the practice environment close to a real proctored assessment, the interview workspace runs in fullscreen mode and records tab switches. The application also offers a practice sandbox, an interview history and a performance dashboard. It is intended as a preparation and self-assessment tool for students and job seekers, not as an automated hiring system.

## 1.2 PROBLEM STATEMENT

Students preparing for technical interviews face several practical problems. One-to-one mock interviews with experienced engineers are expensive and difficult to schedule, so most students practise only a few times before a real interview. Peer mock interviews are more accessible, but the quality of questions and feedback depends heavily on the peer.

Online coding platforms are widely used, but they mainly check whether submitted code passes a set of test cases. They do not ask follow-up questions, do not assess how well a candidate explains a concept or a design trade-off, and do not combine verbal reasoning and coding in a single session. General-purpose chatbots can ask questions, but they do not maintain an interview structure: they may reveal answers, drift off topic, or lose track of how many questions remain.

There is therefore a need for a system that conducts a structured technical interview tailored to the candidate's role and level, asks relevant follow-up questions, evaluates both explanations and code against clear criteria, and returns actionable feedback immediately. The AI-Based Mock Interview Platform addresses this need by combining an LLM interviewer with a controlled interview workflow, validated scoring, an integrated coding workspace and a practice environment.

## 1.3 AIM AND OBJECTIVES

### Aim

To develop an AI-assisted web platform that conducts structured technical mock interviews, evaluates conceptual and coding answers using a large language model, and provides candidates with detailed, skill-wise feedback.

### Objectives

1. To build a full-stack web application (React, TypeScript, Node.js and Express) that supports the complete mock interview lifecycle from configuration to results.
2. To generate interview questions dynamically according to role, experience level, difficulty, technologies and topics using the Google Gemini API.
3. To evaluate answers on technical accuracy, depth, problem solving and communication, and to ask at most one adaptive follow-up question per main question.
4. To provide an integrated coding workspace in which submitted code is reviewed for correctness, algorithm, code quality and time and space complexity.
5. To validate every AI response against a defined schema so that malformed output does not break the interview.
6. To apply proctoring controls during the interview, including fullscreen mode, tab-switch detection and restricted clipboard and developer shortcuts.
7. To present results, interview history, performance analytics and a practice sandbox through a user-friendly interface.

## 1.4 SCOPE OF THE PROJECT

The project covers user registration and login, interview configuration, AI-driven question generation, answer evaluation through text, voice and code, adaptive follow-up questions, proctoring of the interview workspace, final result generation, interview history, a practice sandbox, and profile and settings pages. Ten target roles, five experience levels, four

difficulty levels and seven interview types are supported, along with twenty technologies and eighteen topics.

The quality of generated questions and evaluations depends on the responses of the external LLM service. Scores are produced by the model using the criteria defined in the prompts; they are intended as practice feedback and are not a validated measure for hiring decisions. Code is reviewed by the model rather than executed against hidden test cases; the code execution component is designed as a pluggable provider and currently returns simulated output. Proctoring is implemented in the browser and discourages, but cannot fully prevent, malpractice. The platform requires an internet connection and a valid Gemini API key for its AI features; a built-in mock AI mode is available for offline development.

## 1.5 ORGANISATION OF THE REPORT

Chapter 1 introduces the project, the problem statement, aim, objectives and scope. Chapter 2 reviews existing approaches to automated interviews and AI-based assessment, discusses their limitations, describes the proposed approach and summarises the research papers studied. Chapter 3 presents the system analysis and design, including requirements, architecture, database design, API design and workflow diagrams. Chapter 4 explains the implementation of the main modules. Chapter 5 describes the testing approach, test cases and results, the limitations observed, and screenshots of the system. Chapter 6 concludes the report and outlines future work, followed by the references.

# CHAPTER 2: LITERATURE REVIEW

## 2.1 EXISTING APPROACHES

Several approaches are currently used to prepare for and assess technical interviews.

**Human and peer mock interviews:** Practising with a mentor, senior engineer or classmate gives realistic interaction and personal feedback. Research on personnel selection shows that structured interviews, which use standard questions and scoring rubrics, are considerably more valid predictors of job performance than unstructured ones [1], [2]. However, structured human interviews require trained interviewers and time, which limits how often students can practise.

**Automated interview coaching systems:** Early systems such as MACH used a virtual agent that analysed facial expressions, speech and prosody to give feedback on mock job interviews [3]. Later work predicted expert interview ratings from lexical, prosodic and facial features [4] and used attention-based neural networks on asynchronous video interviews [5]. These systems focus mainly on behavioural and communication cues rather than technical correctness.

**Online coding platforms:** Coding practice websites provide large problem banks and automatically judge submitted code against test cases. They are effective for algorithm practice but evaluate only the final code, not the candidate's reasoning or explanation.

**Large language models:** Transformer-based LLMs [8] trained at scale can answer questions, generate text and understand code [9], [16]. Instruction tuning makes them follow task instructions and a defined persona [10], and models such as GPT-4 and Gemini show strong performance on reasoning and coding tasks [11], [12]. Recent work shows that a strong LLM can act as a judge whose ratings agree with human preferences at a level close to agreement between humans [19], which makes LLM-based answer evaluation feasible.

## 2.2 LIMITATIONS OF EXISTING APPROACHES

1. **Limited access to quality practice:** Expert mock interviews are costly and limited in number, and peer interviews vary widely in quality.
2. **Separation of coding and explanation:** Coding platforms assess code in isolation, while interview coaching systems focus on communication; few tools assess explanation and code together in one session.
3. **No adaptive questioning:** Static question banks cannot ask a follow-up question based on the candidate's answer, which is a key part of real interviews.

4. **Unstructured chatbot sessions:** A general chatbot does not track interview progress, may reveal answers and can return inconsistent or unstructured scores.
5. **Unreliable model output:** LLMs can produce fluent but unsupported content and may return malformed responses [20]; an application that uses the output directly can show wrong scores or fail.
6. **Feedback not linked to practice:** Many tools give a score but no immediate way to practise the weak areas they identify.
7. **Fairness and transparency concerns:** Automated assessment tools used in hiring have been criticised for limited transparency about their validation and bias mitigation [6], [7].

These limitations highlight the need for a single, structured, practice-oriented system that combines adaptive questioning, joint assessment of explanations and code, validated AI output and targeted practice.

## 2.3 PROPOSED APPROACH

The AI-Based Mock Interview Platform uses Google Gemini as the interviewer while the application itself controls the interview structure. The backend stores the configuration and the current question index for every session, generates only one question at a time, and decides when to advance. The model receives a fixed interviewer persona as a system instruction (professional, neutral, does not reveal answers, tests understanding rather than memorisation) and task-specific prompts that require a JSON response with named fields.

Every model response is parsed and validated against a Zod schema with default values before it is used, so the rest of the application always receives well-formed data. Conceptual answers are scored on technical accuracy, depth, problem solving and communication; coding answers are scored on correctness, algorithm, code quality and complexity. A rule in the interview service allows at most one follow-up question per main question and never on the last question, which keeps the session length predictable. At the end of a session the model writes a holistic evaluation of the full history, and the application falls back to averaging the stored per-answer scores if the model is unavailable.

Around this core, the platform provides a proctored workspace, voice and code input, results visualisation, interview history and a practice sandbox. The approach is intended for practice and self-assessment; it supports the learner but does not replace human judgement.

## 2.4 RESEARCH PAPERS

Table 2.1 summarises the research papers studied for this project and how each one relates to the design of the platform.

**Table 2.1: Summary of Literature Survey**

| Sr. No. | Authors (Year) | Approach / Technique | Key Findings | Relevance to This Project |
|---|---|---|---|---|
| 1 | Schmidt and Hunter (1998) [1] | Meta-analysis of 85 years of research on personnel selection methods | Structured interviews had higher predictive validity (0.51) than unstructured interviews (0.38) | Supports a structured interview with fixed criteria for every answer |
| 2 | Hoque et al. (2013) [3] | MACH: virtual agent analysing facial expressions, speech and prosody for mock job interviews | Students who practised with the system improved their interview performance as rated by career counsellors | Shows that automated mock interview practice can improve real interview skills |
| 3 | Naim et al. (2018) [4] | Prediction of expert interview ratings from lexical, prosodic and facial features of recorded mock interviews | What candidates said and how fluently they spoke were strong predictors of ratings | Motivates scoring communication alongside technical content |
| 4 | Hemamou et al. (2019) [5] | HireNet: hierarchical attention model for asynchronous video interviews | Combining text, audio and video improved prediction of hiring recommendations | Basis for a future multimodal (video and audio) extension |
| 5 | Raghavan et al. (2020) [7] | Analysis of the public claims of 18 vendors of algorithmic pre-employment assessments | Vendors disclosed little about how their tools were validated or how bias was handled | The platform is positioned as a practice tool with transparent criteria, not a hiring decision system |
| 6 | Chen et al. (2021) [16] | Codex model evaluated on the HumanEval benchmark of 164 programming problems | Solved 28.8% of problems with one sample and 70.2% with 100 samples | Demonstrates that LLMs can understand and reason about code, enabling AI code review |
| 7 | Ouyang et al. (2022) [10] | Instruction tuning with reinforcement learning from human feedback (InstructGPT) | Outputs of a 1.3B-parameter instruction-tuned model were preferred over those of the 175B GPT-3 | Justifies controlling the model through system instructions (interviewer persona) |

| Sr. No. | Authors (Year) | Approach / Technique | Key Findings | Relevance to This Project |
|---|---|---|---|---|
| 8 | Wei et al. (2022) [13] | Chain-of-thought prompting | Asking for intermediate reasoning steps improved complex reasoning in large models | Informs prompts that ask for reasoning, examples and trade-offs |
| 9 | Zheng et al. (2023) [19] | LLM-as-a-judge evaluated with MT-Bench and Chatbot Arena | A strong LLM judge agreed with human preferences over 80% of the time; position, verbosity and self-enhancement biases were identified | Supports LLM-based answer scoring, while highlighting the need for clear rubrics |
| 10 | Ji et al. (2023) [20] | Survey of hallucination in natural language generation | Models can produce fluent text that is not supported by the input or by facts | Motivates schema validation, default values and fallback results |
| 11 | Kasneci et al. (2023) [21] | Review of opportunities and challenges of LLMs in education | LLMs can personalise learning and feedback but raise concerns about bias, over-reliance and privacy | Frames the platform as a learning aid used with human guidance |
| 12 | Kurdi et al. (2020) [22] | Systematic review of automatic question generation for education | Most systems relied on templates, and evaluating the quality of generated questions remained difficult | Context for using an LLM to generate varied, role-specific questions |

The complete bibliographic details of these papers, together with the documentation of the frameworks and services used, are listed in the References section at the end of this report.

# CHAPTER 3: SYSTEM ANALYSIS AND DESIGN

## 3.1 EXISTING AND PROPOSED SYSTEM

### 3.1.1 Existing System

At present, students prepare for technical interviews by combining several separate resources: question lists and videos for concepts, coding websites for algorithm practice, occasional mock interviews with seniors or peers, and general chatbots for explanations. Each resource covers only one part of the interview. There is no single place where a student can be asked a role-specific question, explain an answer, write code, face a follow-up question and then receive a structured evaluation of the whole session. Feedback is usually delayed, informal or limited to a pass or fail result.

### 3.1.2 Proposed System

The proposed system brings the complete interview experience into one web application. The candidate configures an interview, enters a proctored workspace and answers AI-generated questions by typing, speaking or writing code. Each answer is evaluated immediately against defined criteria, and a follow-up question is asked when the evaluator judges that the answer needs to be probed further. At the end, the candidate receives an overall score, five skill scores, strengths, weaknesses and recommendations, and can continue practising specific topics in the practice sandbox. The interview structure is controlled by the application, while the model provides the questions and judgements, which keeps sessions consistent and predictable.

## 3.2 SYSTEM REQUIREMENTS

### 3.2.1 Hardware Requirements

- Dual-core processor or better.
- 4 GB RAM minimum (8 GB recommended for running the frontend and backend development servers together).
- About 1 GB of free storage for the source code and installed dependencies.
- Stable internet connection for the Gemini API.
- Microphone (optional) for voice answers.

### 3.2.2 Software Requirements

- Operating system: Windows, macOS or Linux.

- Runtime: Node.js 20 or later with npm 10 or later.
- Frontend: React 19, TypeScript, Vite, Tailwind CSS, React Router, Zustand, React Hook Form, Monaco Editor and Recharts.
- Backend: Node.js, Express 4, TypeScript (ES modules), Zod, jsonwebtoken, bcryptjs, Helmet, CORS and express-rate-limit.
- AI service: Google Gemini API through the official Generative AI SDK (model `gemini-3.5-flash`).
- Database: PostgreSQL with Prisma ORM, or the built-in in-memory store for development.
- Browser: a modern browser; Google Chrome or Microsoft Edge is recommended because voice input uses the Web Speech API.
- Development tools: Visual Studio Code and Git.

### 3.2.3 Functional Requirements

- **User authentication:** Allow users to register with name, email and password, log in, and access only their own data.
- **Interview configuration:** Allow the candidate to choose role, experience level, difficulty, interview type, technologies, topics, duration and number of questions.
- **Question generation:** Generate one question at a time for the configured profile, rotating through the selected topics.
- **Answer submission:** Accept answers as typed text, voice transcription or code in one of six languages.
- **Answer evaluation:** Score each answer and return written feedback, strengths and weaknesses.
- **Adaptive follow-up:** Ask a follow-up question when the evaluation indicates that the answer should be explored further.
- **Coding evaluation:** Review submitted code for correctness, algorithm, code quality and time and space complexity.
- **Proctoring:** Lock the workspace in fullscreen, detect tab switches, and restrict copy, paste and developer shortcuts during the interview.
- **Result generation:** Produce an overall score, skill scores, strengths, weaknesses, recommendations and a summary at the end of the interview.
- **History, dashboard and practice:** Show previous sessions, performance analytics and an on-demand practice sandbox.

### 3.2.4 Non-Functional Requirements

- **Usability:** The interface should be simple and consistent, with clear indication of the current question, remaining time and evaluation results.
- **Performance:** Non-AI operations should respond quickly; AI operations depend on the response time of the Gemini API, and the interface should show a waiting indicator meanwhile.
- **Reliability:** Malformed AI responses must not break the interview; responses are validated and missing fields receive default values.
- **Security:** Passwords must be hashed, protected routes must require a valid token, the AI API key must stay on the server, and requests must be validated and rate-limited.
- **Maintainability:** Routes, controllers, services, AI integration and data access should be separated so that each can be changed independently.
- **Extensibility:** The AI provider, database and code execution engine should be replaceable through interfaces.
- **Compatibility:** The application should run on common operating systems and modern browsers.

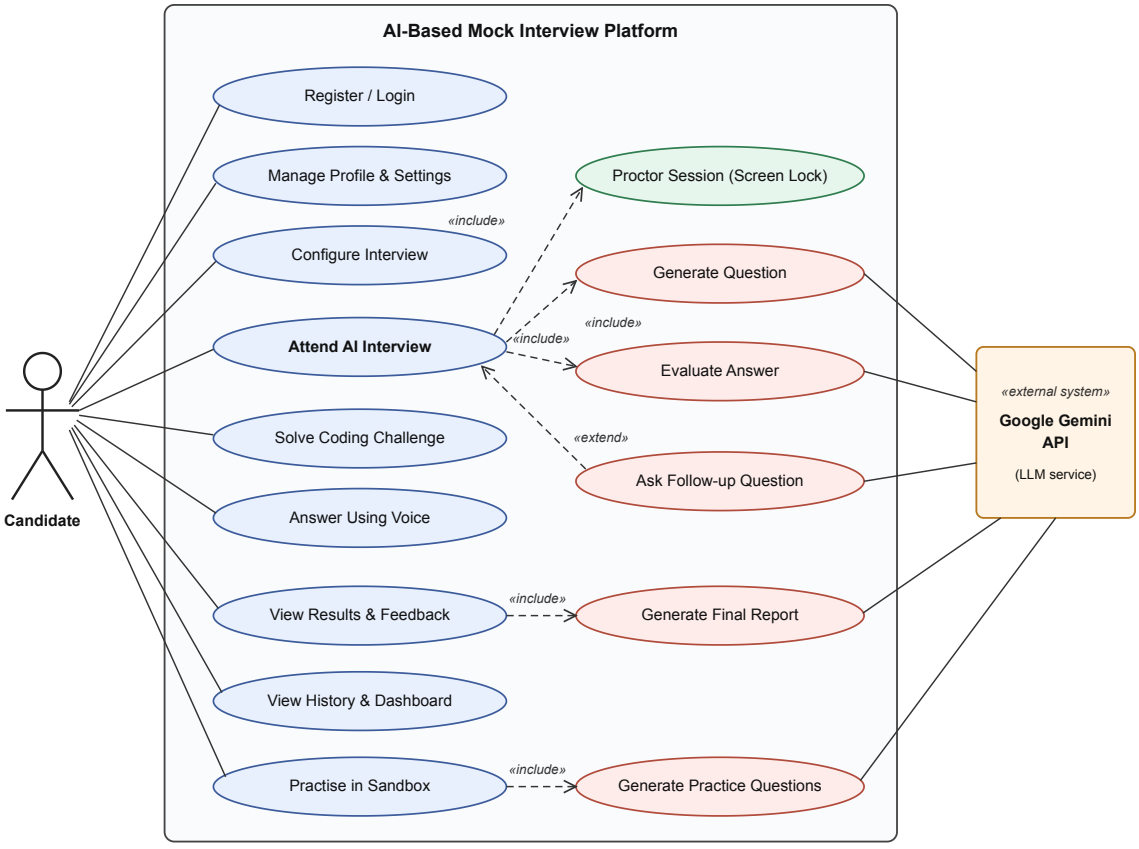



**Figure 3.1: Use Case Diagram of the AI-Based Mock Interview Platform**

Figure 3.1 shows the main use cases. The candidate registers, configures and attends interviews, solves coding challenges, answers by voice, views results and history, and practises in the sandbox. Attending an interview always includes question generation, answer evaluation and proctoring, and may be extended by a follow-up question. The Gemini API is an external system that serves the question, evaluation, follow-up, report and practice generation use cases.

## 3.3 SYSTEM ARCHITECTURE

The platform follows a layered client-server architecture made up of a presentation layer, an application layer, a data layer and external services (Figure 3.2).

**1. Presentation layer:** A React single-page application written in TypeScript and built with Vite. It contains the pages and layouts, Zustand stores for authentication, interview and timer state, the Monaco code editor, the voice recorder, the proctoring hook, Recharts visualisations and forms validated with React Hook Form and Zod. An API client layer sends every request to the backend with the user's JSON Web Token in the Authorization header. During development, Vite forwards all `/api` requests to the backend.

**2. Application layer:** A Node.js and Express server. Each request passes through a middleware pipeline: Helmet security headers, CORS restricted to the frontend origin, a JSON body parser limited to 10 MB, a cookie parser, a rate limiter (100 requests per 15 minutes), JWT authentication for protected routes and Zod validation of request bodies. Routes call controllers, controllers call services, and the services contain the business logic, including the interview state machine and the follow-up rule. The AI service layer exposes an `AIService` interface with two implementations: `GeminiService` for the real model and `MockAIService` for offline development.

**3. Data layer:** A data access facade selects Prisma with PostgreSQL when a `DATABASE_URL` is configured and an in-memory store based on JavaScript Maps otherwise. The Prisma schema defines seven tables.

**4. External services:** The Google Gemini API generates questions, evaluations, follow-ups, final reports and practice questions. The API key is read only on the server and is never sent to the browser. Code execution is designed as a pluggable provider; the current provider returns simulated output, and the design allows Judge0, Piston or a container sandbox to be added later.

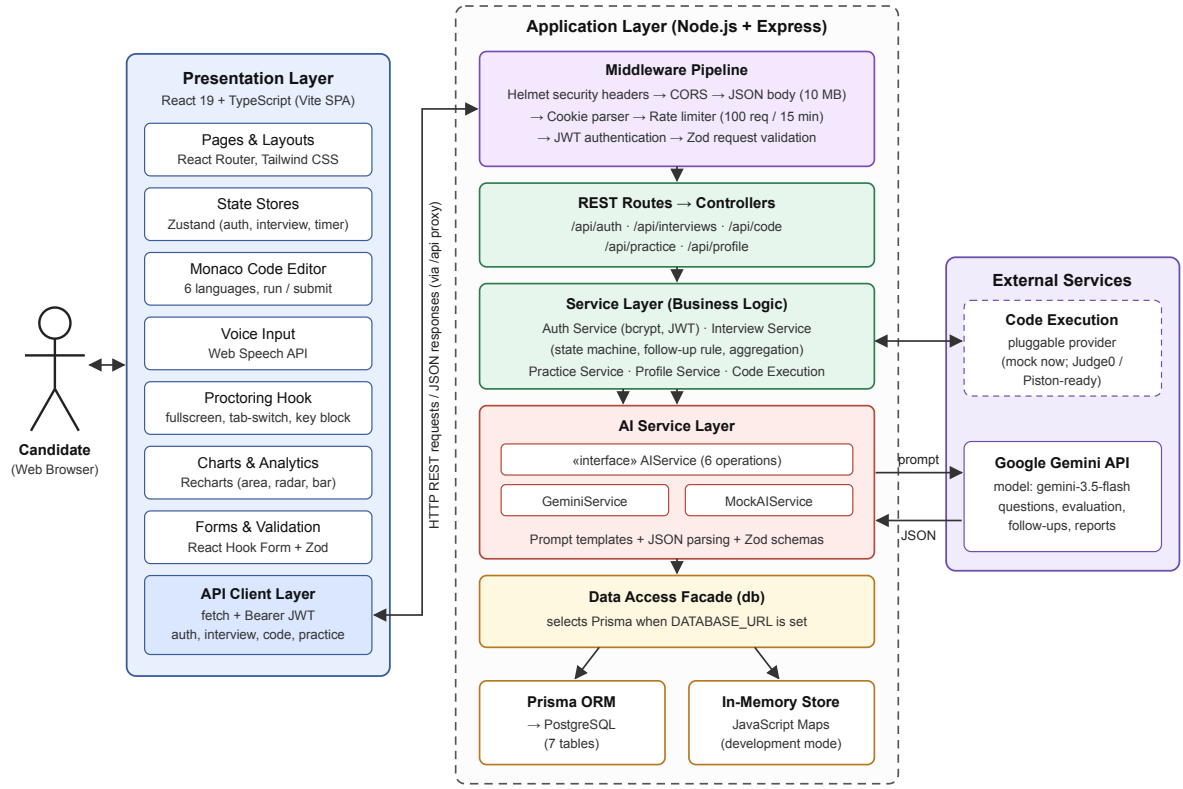



**Figure 3.2: System Architecture of the AI-Based Mock Interview Platform**

This separation keeps the user interface, business rules, AI integration and storage independent of one another. For example, the AI provider can be replaced by implementing the same six-method interface, and the storage can be switched from memory to PostgreSQL by setting one environment variable, without changing the services.

## 3.4 DATABASE DESIGN

The database schema is defined with Prisma for PostgreSQL. All primary keys are UUIDs, and child records are deleted automatically when their parent is deleted (cascade delete). Table 3.1 describes the seven entities.

**Table 3.1: Database Entities and Their Purpose**

| Entity (Table) | Purpose | Important Fields |
|---|---|---|
| User (users) | Registered candidate and profile preferences | email (unique), passwordHash, name, targetRole, experienceLevel, technologies, theme |

| Entity (Table) | Purpose | Important Fields |
|---|---|---|
| Interview (interviews) | One interview session and its configuration and progress | userId, role, difficulty, interviewType, topics, questionCount, status, currentQuestion |
| InterviewQuestion (interview_questions) | A question generated for a session | interviewId, questionNumber, question, type, topic, expectedSkills, codeTemplate |
| CandidateAnswer (candidate_answers) | An answer or follow-up answer with its evaluation | questionId, answer, code, language, score, technicalAccuracy, depth, problemSolving, communication, feedback |
| InterviewResult (interview_results) | Final evaluation of a session (one per interview) | interviewId (unique), overallScore, five skill scores, strengths, weaknesses, recommendations, summary |
| SkillScore (skill_scores) | Optional detailed score for a named skill within a result | resultId, skill, score, maxScore, feedback |
| PracticeQuestion (practice_questions) | Question generated in the practice sandbox and its evaluation | userId, category, difficulty, question, explanation, hints, completed, score |

A user can have many interviews and many practice questions. An interview contains many questions and has at most one result. A question can receive more than one answer, because a follow-up answer is stored against the same question. A result can have many skill scores. Status values of an interview are *pending*, *active* and *completed*, and *currentQuestion* stores the index of the question being answered. Figure 3.3 shows the class structure of the backend, and Figure 3.4 shows the entity-relationship diagram.

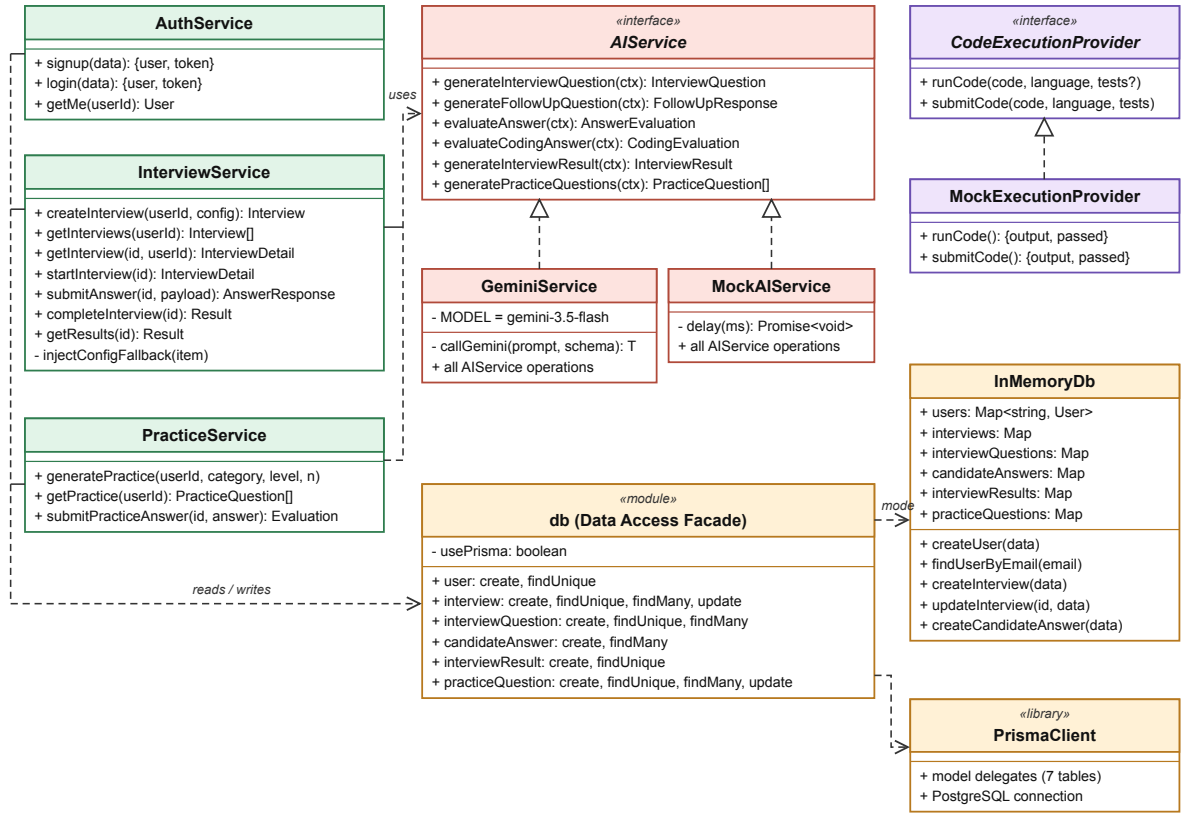



**Figure 3.3: Class Diagram of the Backend Service and AI Layers**

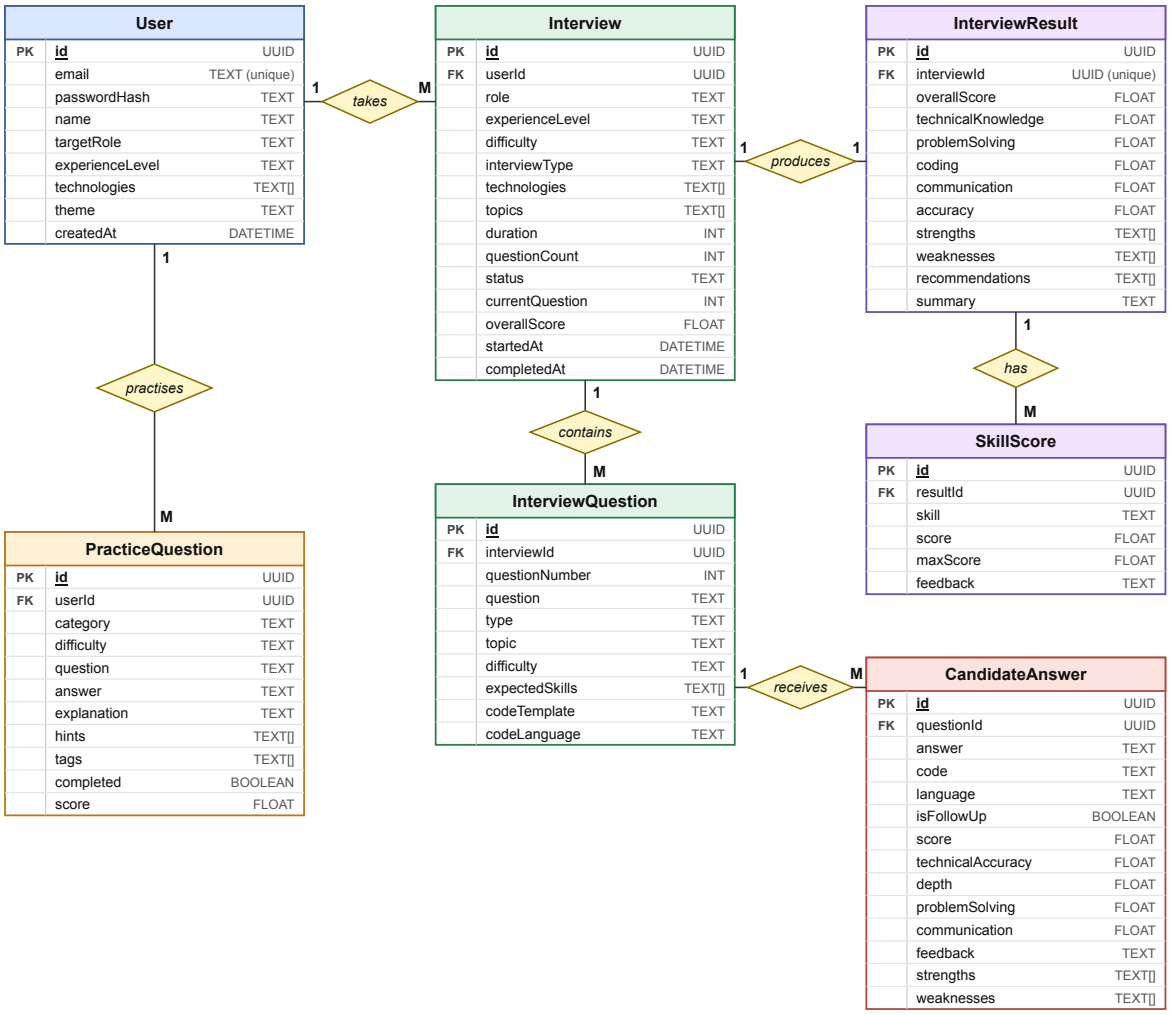



**Figure 3.4: Entity-Relationship Diagram of the Platform Database**

## 3.5 API DESIGN

The backend exposes a REST API under the `/api` prefix (Table 3.2). All routes except signup and login require a valid JSON Web Token, which can be sent as an HTTP-only cookie or as a Bearer token. Request bodies for signup, login and interview creation are validated with Zod schemas, and validation errors are returned with HTTP status 400 and field-level details.

**Table 3.2: REST API Endpoints**

| Method | Endpoint | Description |
|---|---|---|
| POST | `/api/auth/signup` | Register a user and return a token |
| POST | `/api/auth/login` | Log in and return a token |
| GET | `/api/auth/me` | Return the current user |

| Method | Endpoint | Description |
|---|---|---|
| POST | `/api/interviews` | Create an interview session from a configuration |
| GET | `/api/interviews` | List the user's interviews |
| GET | `/api/interviews/:id` | Get a session with its questions and answers; generates the current question if needed |
| POST | `/api/interviews/:id/start` | Activate the session and generate the first question |
| POST | `/api/interviews/:id/answer` | Submit an answer; returns score, feedback and an optional follow-up |
| POST | `/api/interviews/:id/followup` | Follow-up status endpoint (follow-ups are returned by the answer endpoint) |
| POST | `/api/interviews/:id/complete` | Complete the session and generate the result |
| GET | `/api/interviews/:id/results` | Get the stored result, generating it once if absent |
| POST | `/api/code/run` | Run code through the execution provider |
| POST | `/api/code/submit` | Submit code through the execution provider |
| POST | `/api/practice/generate` | Generate practice questions for a category and difficulty |
| GET | `/api/practice` | List the user's practice questions |
| POST | `/api/practice/:id/answer` | Evaluate a practice answer |
| GET / PUT | `/api/profile` | Read or update the profile |

## 3.6 SYSTEM WORKFLOW

The workflow of an interview session is shown in the sequence diagram (Figure 3.5) and the flowchart (Figure 3.6).

1. **Configuration:** The candidate fills in the interview setup form. The backend validates it and stores a new interview with status *pending*.
2. **Start:** The candidate accepts the proctoring rules, which locks the workspace in fullscreen and starts the timer. The backend marks the session *active* and asks Gemini for the first question, using the first selected topic.
3. **Answer and evaluation:** The candidate answers by text, voice or code. The backend sends the question, answer and expected skills (or the code and language) to Gemini

and stores the returned scores and feedback.

4. **Follow-up or next question:** If a follow-up is needed and allowed, the follow-up question is shown and the question index does not change. Otherwise the index is increased and the next question is generated for the next topic.
5. **Completion:** When all questions are answered, or the candidate clicks “Complete Interview”, the timer expires or the third tab-switch violation occurs, the session is completed and Gemini generates the final result from the full history.
6. **Results:** The candidate sees the overall score, skill chart, executive summary, question-by-question review and recommendations.

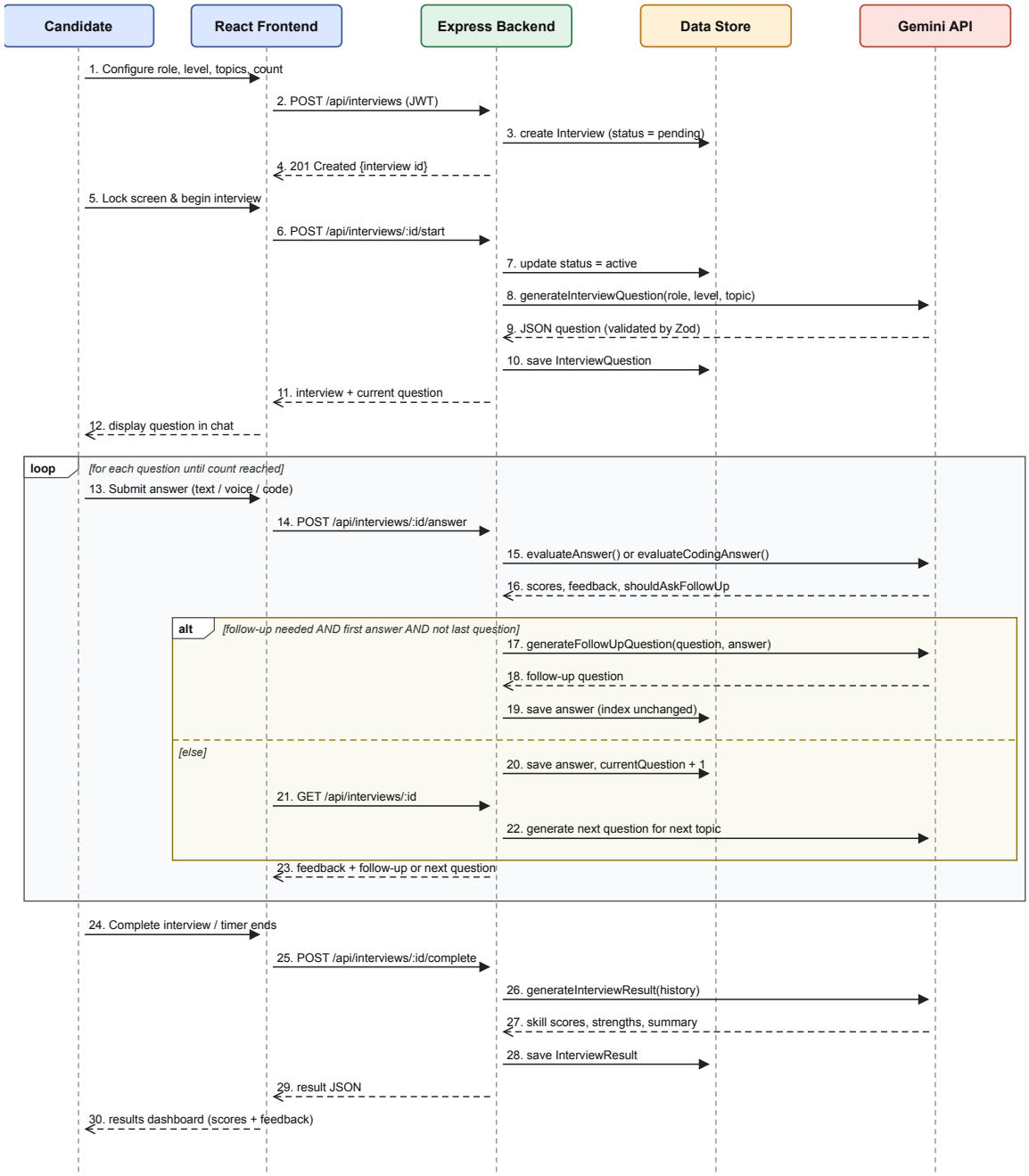



**Figure 3.5: Sequence Diagram of an AI Interview Session**

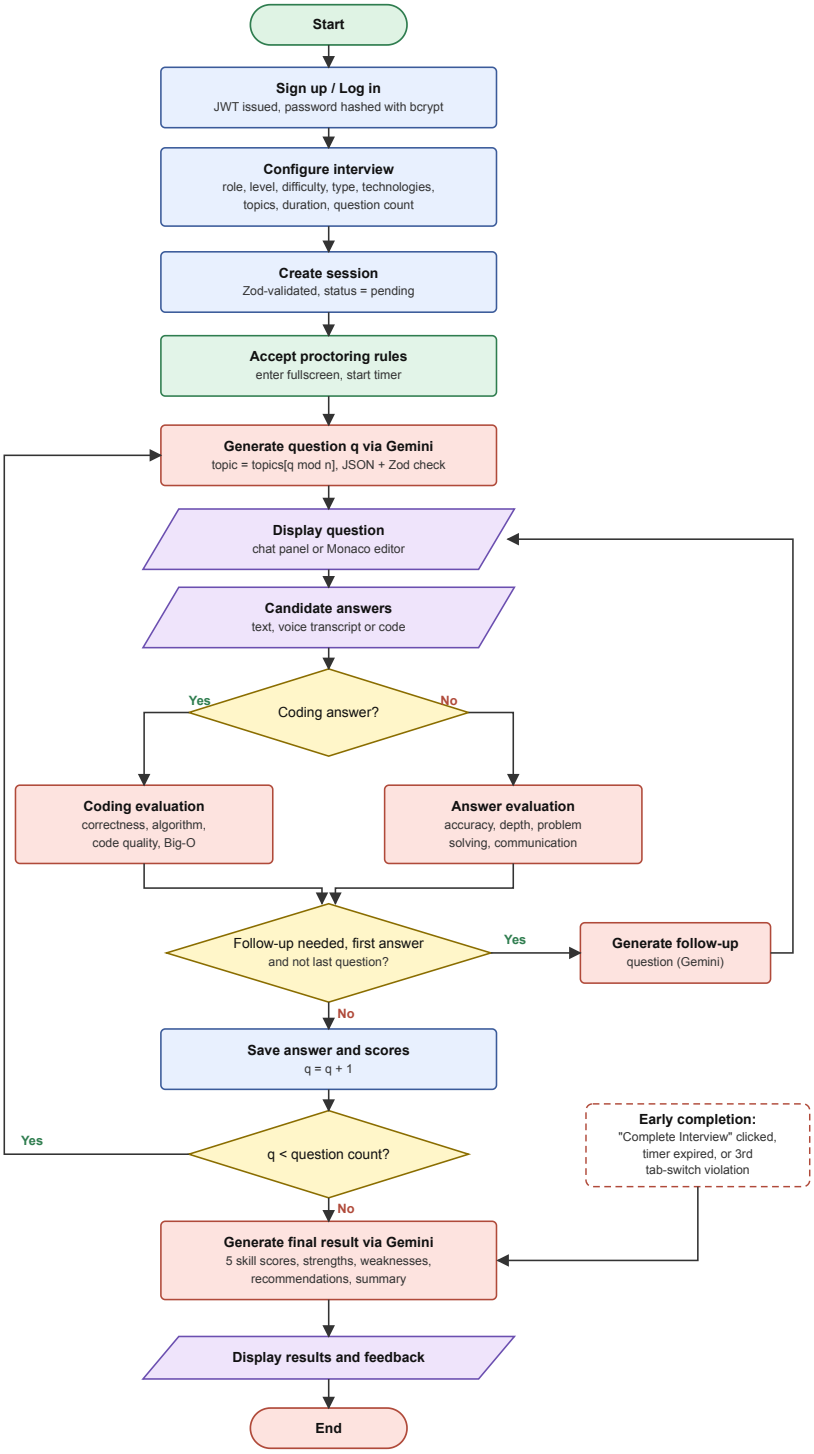



**Figure 3.6: Flowchart of the Interview Process**

# CHAPTER 4: SYSTEM IMPLEMENTATION

## 4.1 TECHNOLOGY STACK

The project is organised as a monorepo with a `client` folder for the React frontend, a `server` folder for the Express backend and a `prisma` folder for the database schema. A root script starts both servers together during development. Table 4.1 lists the main technologies.

**Table 4.1: Technology Stack**

| Layer | Technology | Purpose |
|---|---|---|
| Frontend framework | React 19, TypeScript, Vite | Component-based single-page application and fast development builds |
| Styling and UI | Tailwind CSS 4, Lucide icons, Framer Motion | Responsive dark-themed interface |
| Routing and state | React Router 7, Zustand 5 | Protected routes; authentication, interview and timer stores |
| Forms | React Hook Form with Zod resolver | Client-side validation of signup, login and setup forms |
| Code editor | Monaco Editor (@monaco-editor/react) | Syntax-highlighted editor for six languages |
| Charts | Recharts 3 | Performance, radar and skill bar charts |
| Voice input | Web Speech API | Speech-to-text for spoken answers |
| Backend | Node.js, Express 4, TypeScript (ES modules) | REST API, middleware and business logic |
| AI | Google Gemini API (@google/generative-ai) | Question generation, evaluation, follow-ups and reports |
| Validation | Zod | Validation of request bodies and AI responses |
| Security | jsonwebtoken, bcryptjs, Helmet, CORS, express-rate-limit | Authentication, password hashing, headers and abuse protection |
| Database | Prisma ORM with PostgreSQL; in-memory store | Persistent storage, or zero-setup storage for development |

## 4.2 USER AUTHENTICATION AND SECURITY

On signup, the request body is validated (valid email address, password of at least six characters and a name), the email is checked for uniqueness, and the password is hashed with bcrypt using 10 salt rounds [25]. The server then issues a JSON Web Token [24] containing the user ID, signed with a server-side secret and valid for seven days. The token is returned in the response and also set as an HTTP-only cookie. The frontend stores it and sends it as a Bearer token with every request; on an HTTP 401 response the client clears the token and redirects to the login page.

The authentication middleware accepts the token from the cookie or the Authorization header, verifies it, loads the user and rejects the request with HTTP 401 if any step fails. Responses never include the password hash. Other protections follow common web security practice [26]: Helmet sets security headers such as Content-Security-Policy, Strict-Transport-Security and X-Frame-Options; CORS allows only the configured frontend origin; request bodies are limited to 10 MB; and a rate limiter allows 100 requests per 15 minutes per client. The error handler returns validation details for Zod errors and a generic message for unexpected errors so that internal details are not exposed. The Gemini API key is stored only in the server environment file, and the logger masks values whose keys contain “key”, “secret” or “password”.

## 4.3 INTERVIEW CONFIGURATION

The setup page lets the candidate choose the options shown in Table 4.2. At least one technology must be selected. If no topics are chosen, the selected technologies are used as topics. The configuration is validated again on the server before the interview is created with status *pending* and *currentQuestion* = 0.

**Table 4.2: Interview Configuration Options**

| Parameter | Available Options |
|---|---|
| Target role (10) | Frontend, Backend and Full Stack Developer, DevOps Engineer, Data Engineer, ML Engineer, Mobile Developer, QA Engineer, Security Engineer, Cloud Architect |
| Experience level (5) | Junior, Mid Level, Senior, Staff, Principal |
| Difficulty (4) | Easy, Medium, Hard, Expert |
| Interview type (7) | Technical Screening, Coding Interview, Frontend, Backend, Full Stack, System Design, Algorithms & Data Structures |

| Parameter | Available Options |
|---|---|
| Technologies (20) | React, JavaScript, TypeScript, Node.js, Python, Java, Go, SQL, MongoDB, AWS, Docker, Kubernetes, GraphQL, REST, Redis, PostgreSQL, Vue, Angular, Next.js, C++ |
| Topics (18, optional) | React, JavaScript, TypeScript, Node.js, Python, Algorithms, Data Structures, System Design, Databases, APIs, Performance, Security, Testing, CI/CD, Cloud Architecture, Microservices, Concurrency, Networking |
| Duration | 15, 30, 45 or 60 minutes |
| Question count | 5, 10, 15 or 20 questions |

## 4.4 AI QUESTION GENERATION AND PROMPT ENGINEERING

All model calls go through a single helper in `GeminiService`. It creates a Gemini model client with a fixed **system instruction** that defines the interviewer persona: a senior, professional technical interviewer who must not help the candidate or reveal answers, should ask concise and meaningful questions, test understanding rather than memorisation, ask for examples and trade-offs, keep a neutral tone and evaluate only job-relevant skills without discrimination. Each task prompt then lists the exact JSON fields required, and the helper appends an instruction to respond with valid JSON only [14], [15].

When a question is needed, the backend selects the topic by rotating through the configured topics:

$$topic_q = topics[\ q \bmod n\ ]$$

where $q$ is the zero-based question index and $n$ is the number of topics. The question prompt includes the role, experience level, difficulty, topic and technologies, and asks for the question text, its type (technical, coding or system-design), topic, difficulty, expected skills and optional starter code. In the test session with the topics JavaScript, React and Databases, the first three questions were generated on exactly these topics in order.

The response text is cleaned of any Markdown code fences, parsed as JSON and validated against a Zod schema. The schemas assign safe defaults to optional fields (for example, type defaults to “technical” and expected skills to an empty list), so a partially complete response still produces a usable question, while a response that is not JSON is rejected with a clear error. Questions are generated lazily: only the current question is created, and it is created only when the session is active and the question does not already exist. This keeps the number of model calls to what the session actually needs.

## 4.5 ANSWER EVALUATION AND ADAPTIVE FOLLOW-UP

For a conceptual answer, the evaluation prompt contains the question, the candidate's answer, the expected skills and, for a follow-up answer, the earlier conversation on the same question. The model returns the fields listed in Table 4.3, all scored from 0 to 100, along with feedback, strengths, weaknesses and a flag that indicates whether a follow-up question should be asked.

**Table 4.3: Answer Evaluation Criteria**

| Field | Range / Type | Meaning |
|---|---|---|
| score | 0 – 100 | Overall quality of the answer |
| technicalAccuracy | 0 – 100 | Correctness of the technical content |
| depth | 0 – 100 | Depth of understanding beyond a surface definition |
| problemSolving | 0 – 100 | Quality of reasoning and approach |
| communication | 0 – 100 | Clarity and structure of the explanation |
| feedback | text | Constructive feedback paragraph |
| strengths, weaknesses | list | Specific positive points and gaps |
| shouldAskFollowUp | boolean | Evaluator's recommendation to probe further |

The decision to ask a follow-up is made by the interview service, not by the model alone. Let $E$ be the evaluator's follow-up flag, $L$ be true when the current question is the last one, and $k$ be the number of answers already stored for the current question. A follow-up is requested only when

$$F = E \wedge \neg L \wedge (k = 0)$$

If $F$ is true, the service asks the model to write a follow-up question based on the original question and answer. If the follow-up call fails, a default follow-up asking about trade-offs and alternative approaches is used. The answer is stored with the follow-up text appended to its feedback, and the question index is **not** advanced, so the next answer is treated as the follow-up answer and is evaluated with the earlier exchange as context. When $F$ is false, the index is advanced; if the new index equals the question count, the session is marked completed. As a result, each main question receives at most one follow-up, no follow-up is asked on the last question, and the total number of interactions is bounded by twice the question count. Figure 4.1 shows these states and transitions.

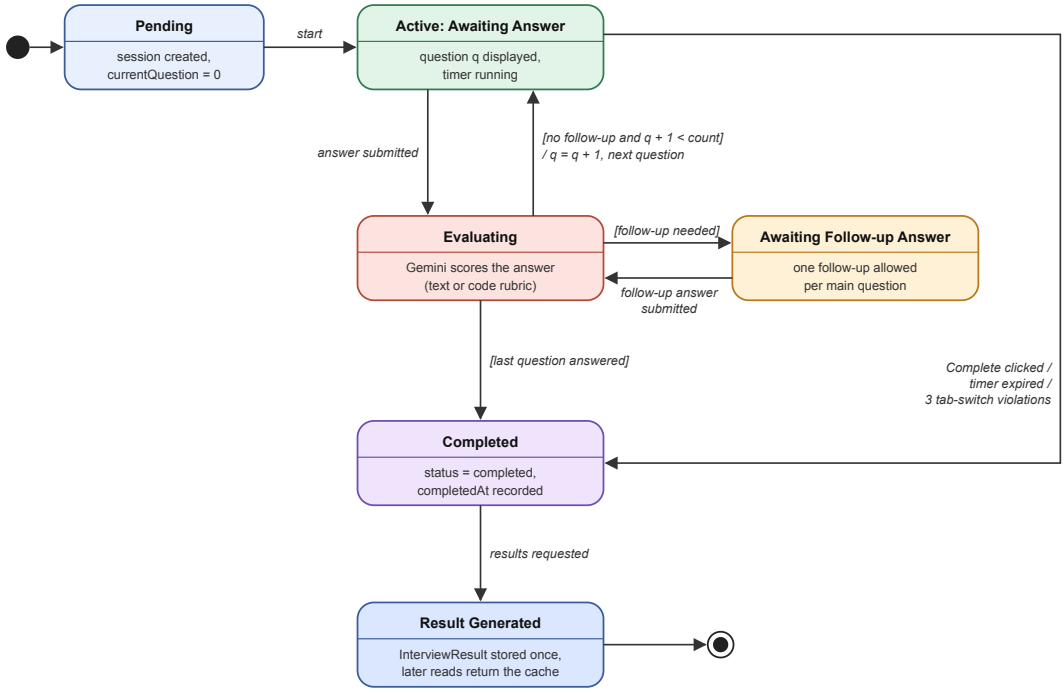



**Figure 4.1: State Diagram of an Interview Session**

## 4.6 CODING CHALLENGE EVALUATION

When the generated question is of type "coding", the workspace switches to the Monaco editor and loads the starter code supplied by the model. The candidate can choose JavaScript, TypeScript, Python, Java, C++ or Go, reset the editor, run the code and submit it. The Run button calls the code execution API; the current provider returns a simulated result ("Mock execution successful"), and the provider interface is designed so that a real sandbox such as Judge0 or Piston can be connected later.

On submission, the backend sends the problem, language and code to the model with the coding evaluation prompt [16], [19]. The model returns a score, correctness, algorithm and code quality ratings (0–100), the time and space complexity in Big-O notation, feedback, strengths and improvements. These values are mapped to the common answer fields as shown in Table 4.4, so that coding and conceptual answers can be aggregated together. Coding answers do not trigger follow-up questions.

**Table 4.4: Mapping of Coding Evaluation to Answer Scores**

| Coding Evaluation Field | Stored Answer Field | Notes |
|---|---|---|
| score | score | Overall coding score |
| correctness | technicalAccuracy | Does the code solve the problem |

| Coding Evaluation Field | Stored Answer Field | Notes |
|---|---|---|
| codeQuality | depth | Readability, structure, edge cases |
| algorithm | problemSolving | Choice and efficiency of approach |
| (not assessed) | communication = 80 | Fixed value for code submissions |
| timeComplexity, spaceComplexity | appended to feedback | Shown to the candidate, e.g. O(N) |
| improvements | weaknesses | Suggested improvements |

## 4.7 RESULT GENERATION AND SCORE AGGREGATION

When a session is completed, the service collects every question with its answers, scores and feedback and asks the model for a holistic result containing the overall score; the technical knowledge, problem solving, coding, communication and accuracy scores (0–100); strengths; weaknesses; recommendations; and a summary. Because the model sees the whole history, it can take into account questions that were not answered, which a simple average of answered questions would ignore. The result is stored once per interview; later requests return the stored result.

If the model call fails, the service computes the result from the stored answers. For $n$ stored answers with scores $s_i$, technical accuracy $a_i$, problem solving $p_i$ and communication $c_i$:

$$Overall = round(\Sigma\, s_i / n), \quad Technical\ Knowledge = Accuracy = round(\Sigma\, a_i / n)$$

$$Problem\ Solving = round(\Sigma\, p_i / n), \quad Communication = round(\Sigma\, c_i / n)$$

$$Coding = round(\Sigma\, k_i / n), \quad where\ k_i = s_i\ if\ answer\ i\ contains\ code,\ otherwise\ a_i$$

If there are no answers, every score defaults to 60. This fallback was exercised during testing: for a session with one coding answer scored $s$ = 98, $a$ = 100, $p$ = 100 and $c$ = 80, the stored result was Overall 98, Technical Knowledge 100, Problem Solving 100, Coding 98, Communication 80 and Accuracy 100, which matches the formulas above.

**Illustrative calculation.** For two coding answers with scores 100 and 92, technical accuracy 100 and 95, problem solving 100 and 90, and communication 80 and 80, the fallback gives Overall = round(192 / 2) = 96, Technical Knowledge = round(195 / 2) = 98, Problem Solving = 95, Coding = 96 and Communication = 80. In the actual test session these were the first two answers of a five-question interview, and the model-generated result was Overall 64

because the third question had not been answered. This shows the advantage of the holistic evaluation; the fallback is used only when the model is unavailable.

The results page shows the overall score as a ring, the five skill scores as a bar chart, the executive summary, a question-by-question review with the candidate's answer, score and feedback, and lists of strengths, weaknesses and recommendations.

## 4.8 PROCTORING AND SCREEN LOCK

Before the interview begins, the candidate sees the proctoring guidelines and must click "Lock Screen & Begin Interview". A custom React hook then activates the rules in Table 4.5 using the browser's Fullscreen, Page Visibility, Clipboard and History APIs [34]. The rules are removed automatically when the interview is completed or the timer expires.

**Table 4.5: Proctoring Rules in the Interview Workspace**

| Rule | Behaviour |
|---|---|
| Fullscreen lock | The page requests fullscreen and requests it again if the candidate leaves fullscreen |
| Tab-switch detection | Each time the page becomes hidden, a warning overlay is shown with the count ("Warning 1 of 3") |
| Maximum violations | On the third tab switch the interview is completed automatically and the results are generated |
| Clipboard restriction | Copy and cut events and the right-click context menu are cancelled |
| Shortcut restriction | Ctrl/Cmd + C, V, X, P, S, U, I; F12; and Ctrl/Cmd + Shift + I, J, C are blocked |
| Navigation guard | The browser Back button is neutralised and a confirmation is shown before closing or refreshing the page |
| Session timer | Countdown with warning colours in the last five minutes and the last minute; on expiry the interview is completed |

These controls are implemented in the browser and are meant to discourage casual malpractice, such as copying questions or searching in another tab. They cannot prevent the use of a second device, and they are not a substitute for supervised proctoring.

## 4.9 VOICE INPUT AND PRACTICE SANDBOX

**Voice input:** The interview workspace has three modes: text, code and voice. In voice mode, the recorder uses the browser's SpeechRecognition interface [33] in continuous mode with interim results and the English (US) language setting. Final transcription segments are

appended to the answer text, which the candidate can review before submitting. If the browser does not support speech recognition, the recorder shows that voice input is unavailable.

**Practice sandbox:** The practice page offers twelve categories (JavaScript, TypeScript, React, Node.js, Python, Algorithms, Data Structures, SQL, System Design, APIs, Docker and Cloud) and three difficulty levels. Clicking “Generate Questions” asks the model for three practice questions with hints and a model solution. When the candidate submits an answer, it is evaluated with the same evaluation prompt as interview answers, and the score and feedback are stored with the practice question, which is then marked as completed. This links the weaknesses identified in an interview to immediate, focused practice.

## 4.10 DASHBOARD AND INTERVIEW HISTORY

The dashboard shows four summary cards (interviews completed, average score, best score and technical score), a performance-over-time area chart, a category radar chart, recent sessions and a targeted practice recommendation. The average is the mean score of completed sessions, the best score is their maximum, and the technical score is computed as min(100, round(1.05 × average)). The history page lists sessions with role, date, duration, interview type, difficulty and score, and can be filtered by role and difficulty. For a new account with no completed sessions, both pages display built-in sample data so that the layout can be understood before the first interview. The current limitations of these statistics are discussed in Section 5.4.

## 4.11 DEVELOPMENT MOCK AI MODE

All AI operations are defined by the six-method `AIService` interface. At start-up the server uses `GeminiService` when a valid `GEMINI_API_KEY` is present and `USE_MOCK_AI` is not “true”; otherwise it uses `MockAIService`, which returns fixed, realistic responses after a short delay. This lets the whole interface be developed and demonstrated without an API key or network access, and it allows automated tests to run without calling the real model. Similarly, the server uses the in-memory store when no database URL is configured, so the application can run with no setup beyond installing dependencies.

# CHAPTER 5: TESTING AND RESULTS

## 5.1 TESTING APPROACH

Testing was carried out on the running application with the frontend on the Vite development server, the backend on Node.js, the in-memory data store and the real Gemini model (`gemini-3.5-flash`). Three kinds of testing were performed:

- **API functional testing:** Requests were sent directly to the REST endpoints to check authentication, validation, access control, interview creation, code execution and security headers, and the HTTP status and response body were recorded.
- **End-to-end testing:** A browser was used to register, configure interviews, answer AI-generated questions, complete sessions and view results, history and practice pages, as a candidate would.
- **Proctoring testing:** Keyboard, clipboard and context-menu events were triggered in the locked workspace to check whether they were blocked, and tab switches were performed to check the warnings and the automatic completion.

The actual result of every test case was recorded as observed, including cases where the behaviour differed from what was expected.

## 5.2 TEST CASES AND RESULTS

**Table 5.1: Functional Test Cases and Results**

| ID | Test Case | Expected Result | Actual Result | Status |
|---|---|---|---|---|
| TC01 | Register with valid details | Account created; token returned; no password in response | HTTP 201; token returned; user object contains only id, email and name | Pass |
| TC02 | Register with invalid email | Request rejected with validation error | HTTP 400 “Validation Error”, field email: “Invalid email” | Pass |
| TC03 | Register with a 3-character password | Rejected (minimum 6 characters) | HTTP 400; “String must contain at least 6 character(s)” | Pass |
| TC04 | Register with an existing email | Rejected with a conflict message | Rejected, but with HTTP 500 and a generic “Internal Server Error” message | Pass (status code to refine) |
| TC05 | Login with correct credentials | Token returned | HTTP 200; token returned | Pass |

| ID | Test Case | Expected Result | Actual Result | Status |
|---|---|---|---|---|
| TC06 | Login with wrong password | Rejected as unauthorised | Rejected, but with HTTP 500 and a generic message | Pass (status code to refine) |
| TC07 | Protected API without token | HTTP 401 | HTTP 401 “Unauthorized” | Pass |
| TC08 | Protected API with tampered token | HTTP 401 | HTTP 401 “Invalid token” | Pass |
| TC09 | Create interview with missing fields | HTTP 400 with field errors | HTTP 400 listing each missing field | Pass |
| TC10 | Create interview with valid configuration | Session created as pending | HTTP 201; status “pending”; currentQuestion 0 | Pass |
| TC11 | Start interview | First question generated for the first topic | Gemini generated a JavaScript coding question on concurrent Promise handling with starter code | Pass |
| TC12 | Topic rotation | Question q uses topics[q mod n] | Questions 1–3 were on JavaScript, React and Databases, as configured | Pass |
| TC13 | Submit coding answer | Score, feedback, time and space complexity | 100/100 with detailed feedback and complexity; next question generated | Pass |
| TC14 | Submit second coding answer | Evaluation and next question | 92/100 (accuracy 95, quality 95, algorithm 90) | Pass |
| TC15 | Complete interview early | Result with skill scores, strengths, weaknesses, recommendations, summary | Overall 64/100; summary and weaknesses mentioned the unanswered SQL question | Pass |
| TC16 | Request result again | Same stored result returned | Same result record returned after page reload | Pass |
| TC17 | Result when AI is unavailable | Averages of stored answer scores | 98 / 100 / 100 / 98 / 80 / 100, matching the fallback formulas | Pass |
| TC18 | Run code | Execution output shown | “Mock execution successful” shown in the output terminal | Pass (simulated) |
| TC19 | Blocked shortcuts in locked mode | Ctrl/Cmd+C, Cmd+V, F12, Ctrl+Shift+I, right-click and copy blocked; normal typing allowed | All six actions cancelled; ordinary key press not blocked | Pass |

| ID | Test Case | Expected Result | Actual Result | Status |
|---|---|---|---|---|
| TC20 | Tab switch during interview | Warning with violation count | Overlay “Warning 1 of 3”; header shows “1/3 warnings” | Pass |
| TC21 | Third tab switch | Interview completed automatically | Interview completed and redirected to the results page | Pass |
| TC22 | Security headers and rate limit | Helmet headers and 100 requests / 15 min policy | CSP, HSTS, X-Frame-Options, X-Content-Type-Options and “RateLimit-Policy: 100;w=900” present | Pass |
| TC23 | Follow-up on a text answer | Follow-up asked when the evaluator requests it | Not executed: the daily Gemini API quota was exhausted before a text-answer session could be completed | Not tested |
| TC24 | Gemini temporarily unavailable (HTTP 503 / 429) | Clear message and retry | Request failed with the generic message “An error occurred”; no automatic retry | Fail |
| TC25 | Dashboard after real sessions | Average and best scores from results | Showed 2 completed sessions with average and best score 0/100 | Fail |

Of the 25 test cases, 22 passed (two of them with a remark about the HTTP status code and one with simulated execution), one could not be executed because of the API quota, and two failed. The failures concern error handling for the external AI service and the dashboard statistics; both are described in Section 5.4.

## 5.3 RESULTS AND DISCUSSION

A five-question *Full Stack Developer* interview (Junior level, Medium difficulty, Technical Screening, 30 minutes) was conducted with the topics JavaScript, React, Databases, APIs and Algorithms. Table 5.2 shows the evaluation returned by the model for the questions reached before the session was completed early.

**Table 5.2: Observed AI Evaluation in the Sample Session**

| Q | Question (topic, type) | Score | Accuracy | Depth / Quality | Problem Solving | Commu-nication | Complexity reported |
|---|---|---|---|---|---|---|---|
| 1 | Combine three async calls with concurrency and fallbacks (JavaScript, coding) | 100 | 100 | 100 | 100 | 80* | O(1) / O(1) |

| Q | Question (topic, type) | Score | Accuracy | Depth / Quality | Problem Solving | Commu-nication | Complexity reported |
|---|---|---|---|---|---|---|---|
| 2 | Custom React hook that avoids stale responses (React, coding) | 92 | 95 | 95 | 90 | 80* | O(1) / O(1) |
| 3 | N+1 query problem and a single efficient SQL query (Databases, technical) | Not answered (session completed early) | | | | | |

* Communication is fixed at 80 for code submissions (Table 4.4).

The model-generated final result for this session was: Overall 64, Technical Knowledge 65, Problem Solving 65, Coding 68, Communication 85 and Accuracy 64. The strengths it listed were accurate (asynchronous patterns and Promise combinators, race-condition handling with effect cleanup, clean code), and the weaknesses correctly noted the missing answer to the SQL question and suggested using an `AbortController` to cancel stale requests, which was a valid improvement to the submitted React hook. The recommendations (study relational joins and the N+1 problem; adopt AbortController) followed directly from these weaknesses.

These observations show that the platform produced role- and topic-specific questions, gave scores that reflected the quality of the answers, and wrote feedback that was specific to the submitted code rather than generic. Three points should be noted when interpreting the results. First, the scores are judgements of the language model and are not calibrated against human interviewers; the same answer may receive slightly different scores in different runs. Second, complexity analysis is produced by the model without running the code; for question 1 it reported O(1), which is reasonable for a fixed number of calls but is not derived from execution. Third, the number of sessions tested was small because of the daily request quota of the API key used, so the results demonstrate functionality rather than statistical accuracy.

## 5.4 SYSTEM LIMITATIONS

1. **Dependence on the external AI service:** Question generation, evaluation and reports require the Gemini API. During testing the API returned HTTP 503 (“high demand”) several times and later HTTP 429 (daily quota exceeded). The application has no automatic retry and shows only a generic error message; if the failure occurs while the next question is being fetched, the interface stays on the previous question until the page is refreshed.
2. **Model-based scoring:** Scores and feedback are produced by an LLM and may vary between runs or reflect model biases [19], [20]. They are suitable for practice feedback,

not for hiring decisions.

3. **Simulated code execution:** The Run and Submit buttons use a mock execution provider. Code is reviewed by the model, not executed against hidden test cases.
4. **Dashboard statistics:** The dashboard computes averages from a score field of the interview record that is not updated when the result is generated, so completed sessions showed 0/100. Dates in the dashboard and history appeared as “Invalid Date” with the in-memory store, which does not set creation timestamps. For new accounts, the dashboard and history show sample data, and the category radar chart always uses sample values.
5. **Observed interface issues:** In development mode, the first question was generated twice in one trial because the start request was sent twice (React Strict Mode runs effects twice); a server-side guard would prevent this. The results page appeared blank immediately after an automatic redirect from the interview workspace and displayed correctly after a refresh. Reloading an active interview page displays the first question again. The countdown timer starts at 45 minutes regardless of the duration selected during setup.
6. **Browser-based proctoring:** Proctoring rules can be bypassed with a second device or by disabling scripts, and voice input depends on browser support for the Web Speech API.
7. **Error status codes:** Duplicate registration and wrong-password login are rejected correctly but return HTTP 500 instead of 409 and 401.
8. **Persistence and profile:** The tested configuration used the in-memory store, so data is lost when the server restarts. The profile service currently returns the submitted data without storing it.

None of these limitations affects the core interview flow when the AI service is available, and most of them can be addressed with small, targeted changes, as described in Section 6.2.

## 5.5 SYSTEM SCREENSHOTS

The following screenshots were captured from the running application. Screens that involve AI output show real responses from the Gemini model.

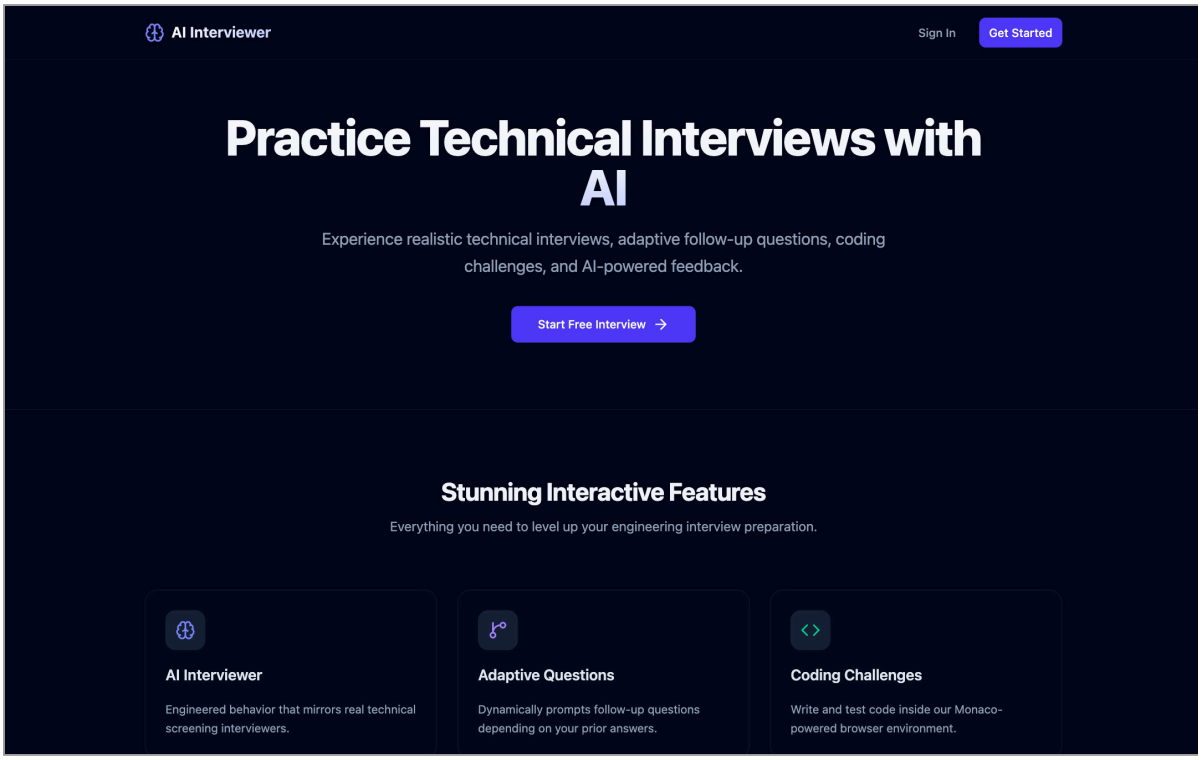



**Figure 5.1: Landing Page**

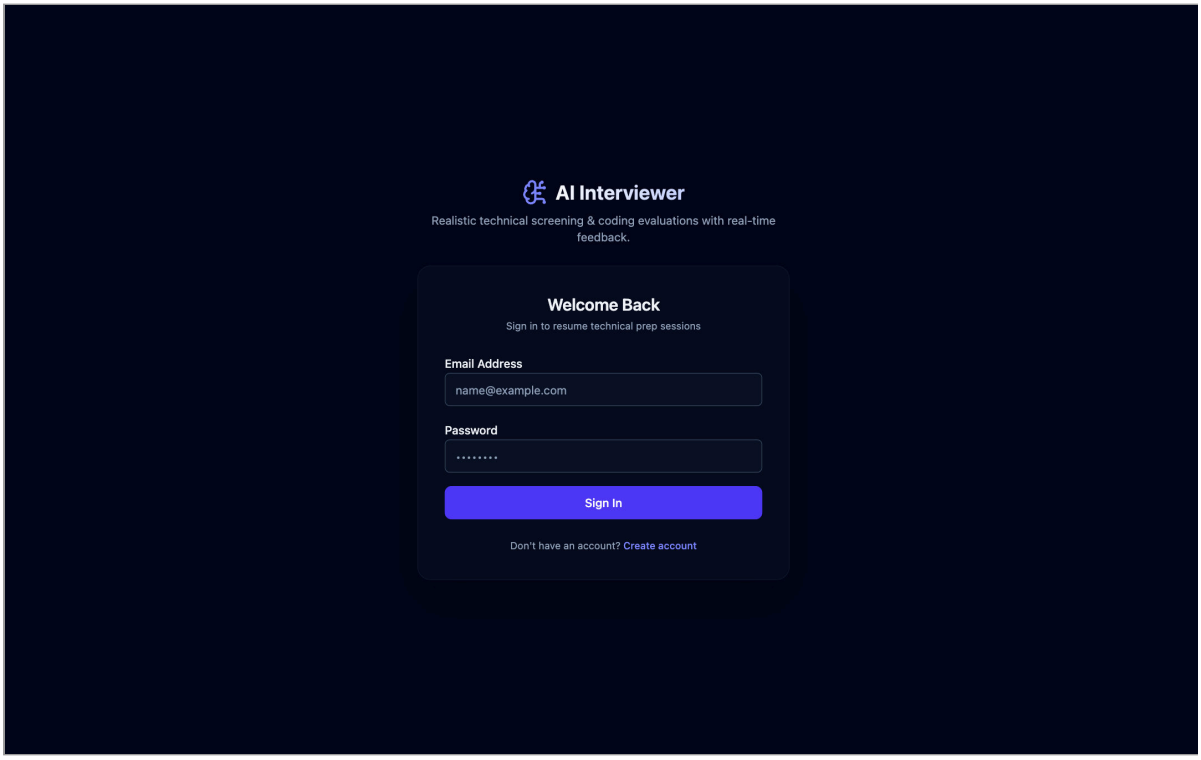



**Figure 5.2: Login Page**

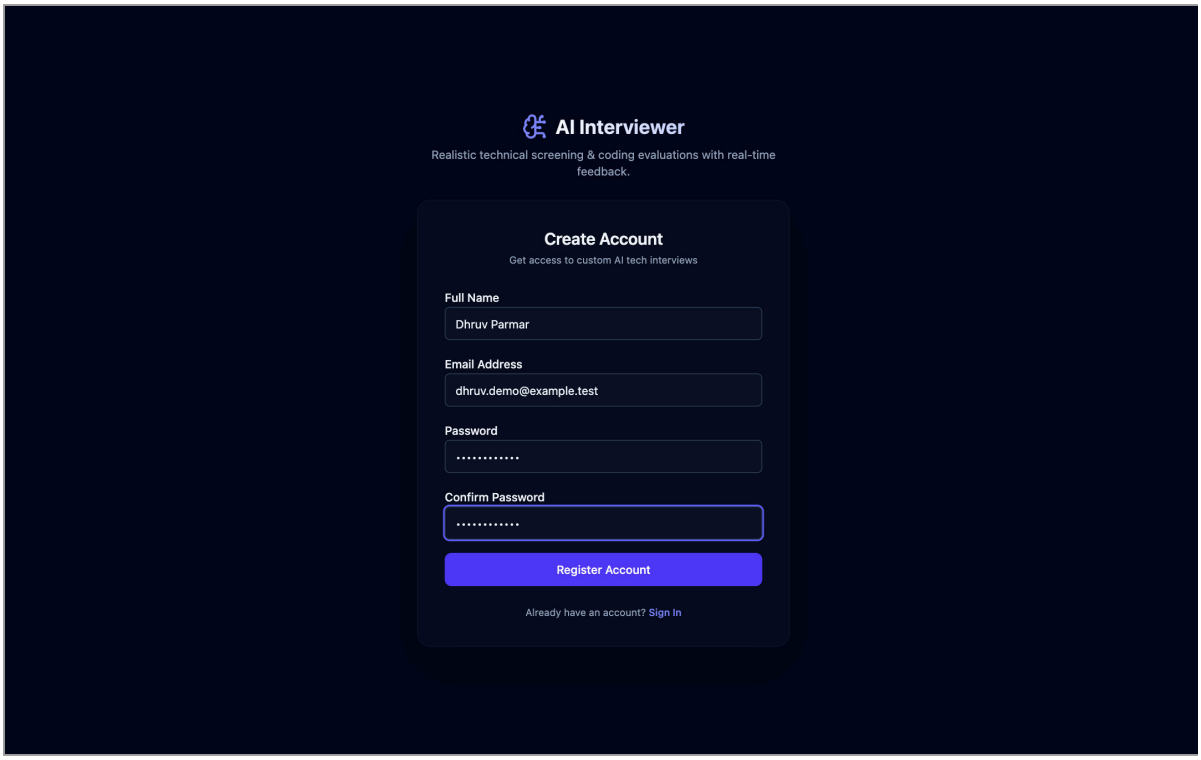



**Figure 5.3: Sign-up Page**

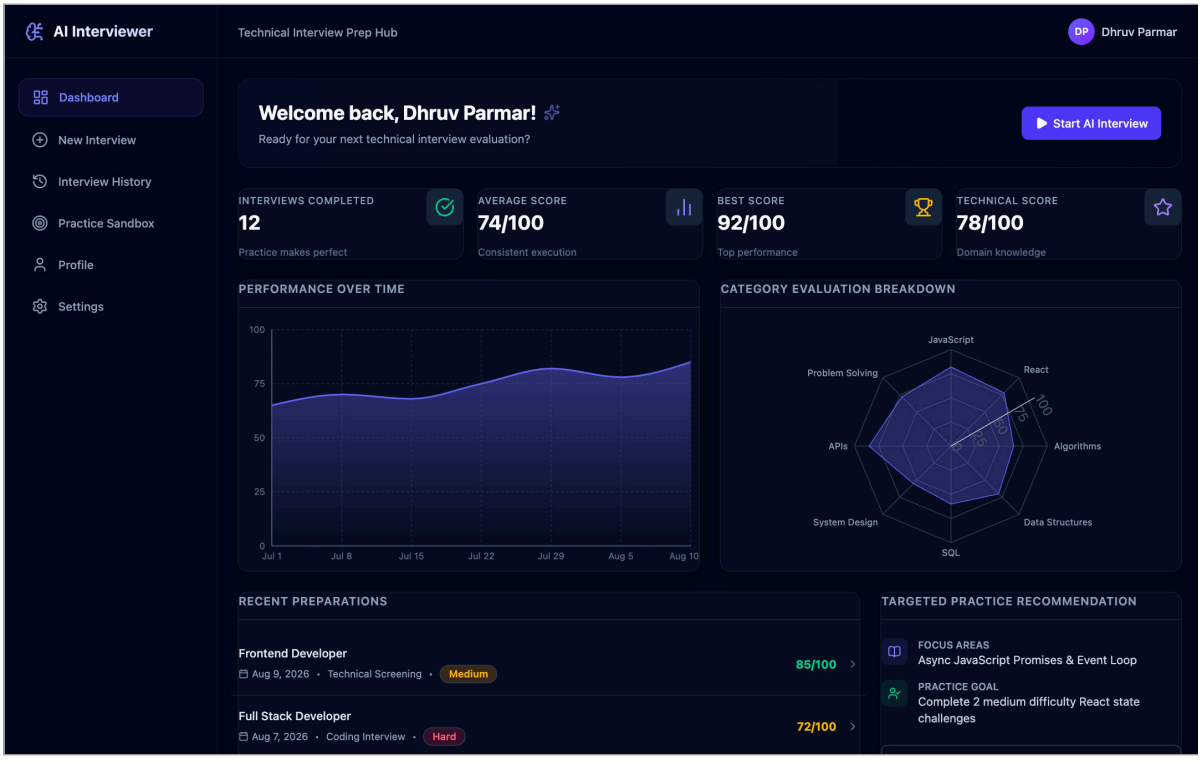



**Figure 5.4: Candidate Dashboard (Sample Data for a New Account)**

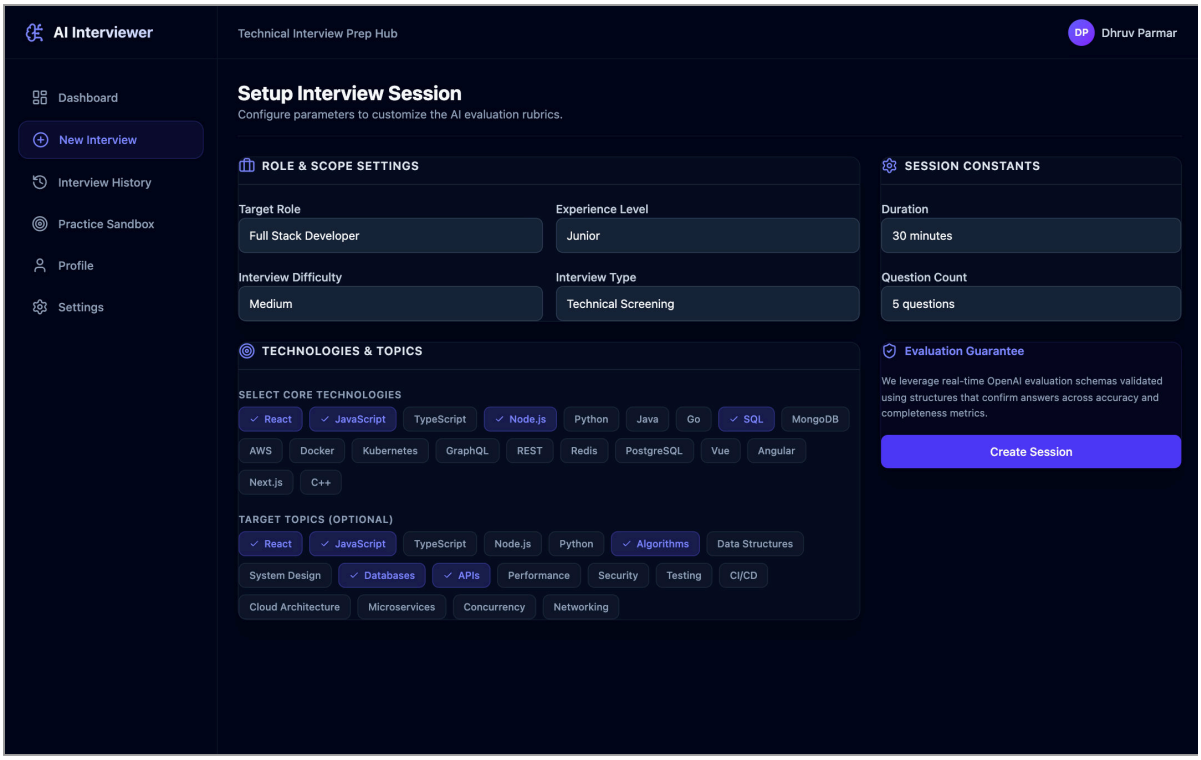



**Figure 5.5: Interview Setup Page**

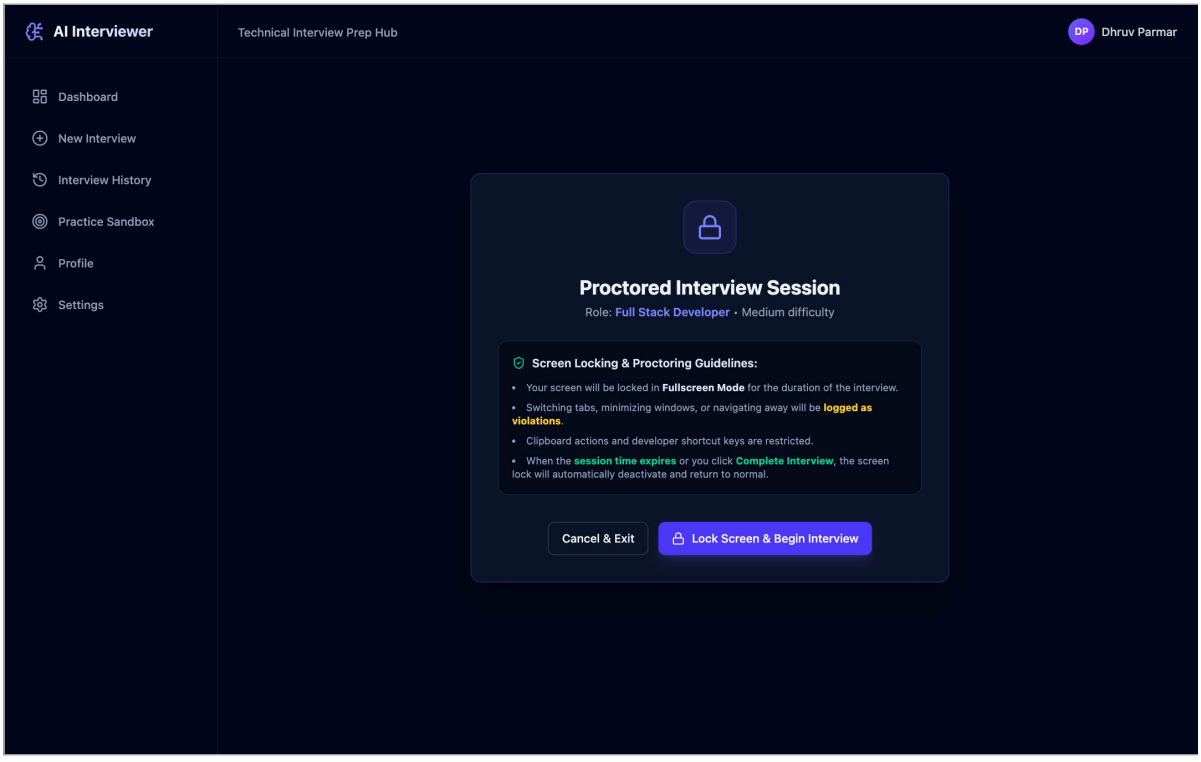



**Figure 5.6: Proctored Session Start Screen**

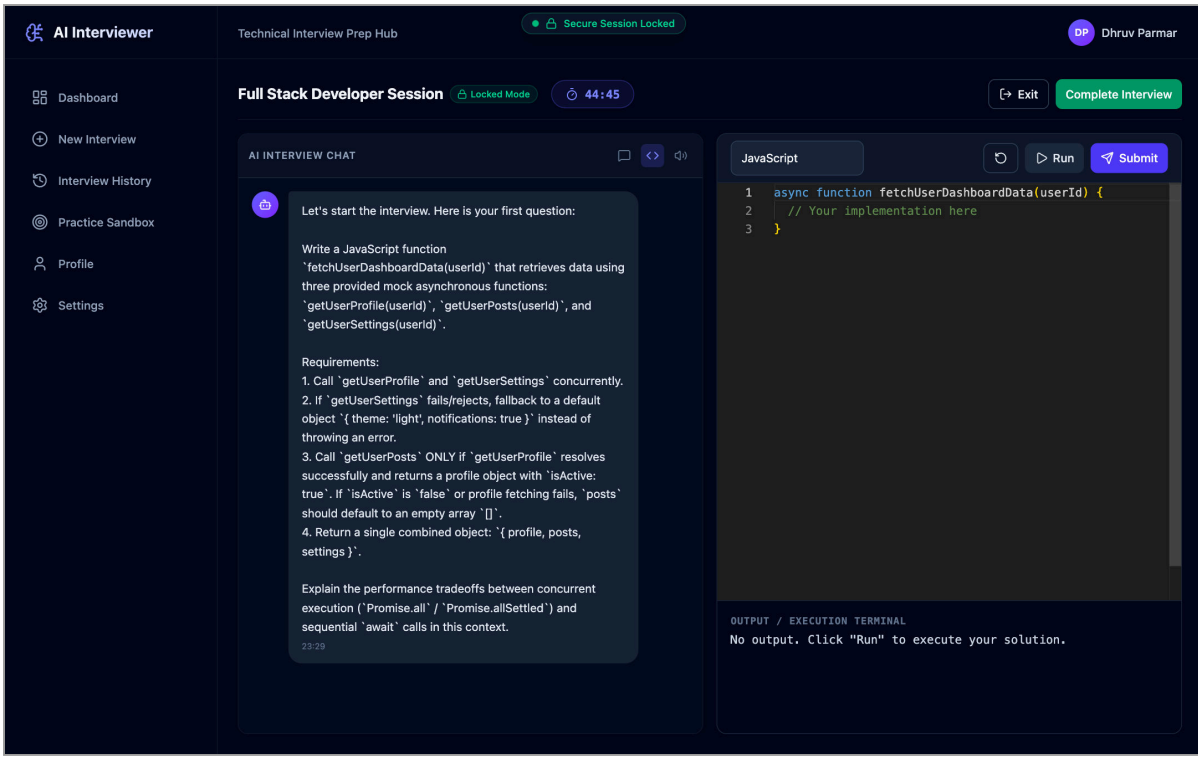



**Figure 5.7: AI-Generated Coding Question in the Interview Workspace**

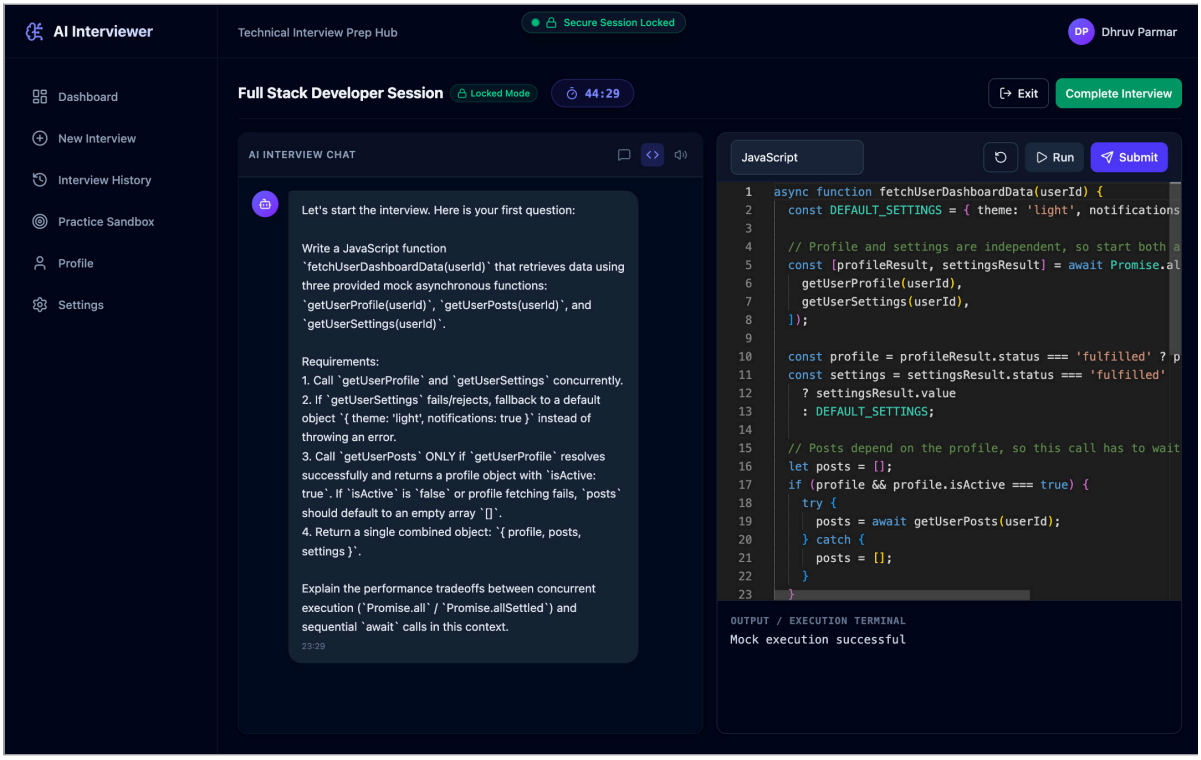



**Figure 5.8: Code Editor with Execution Terminal**

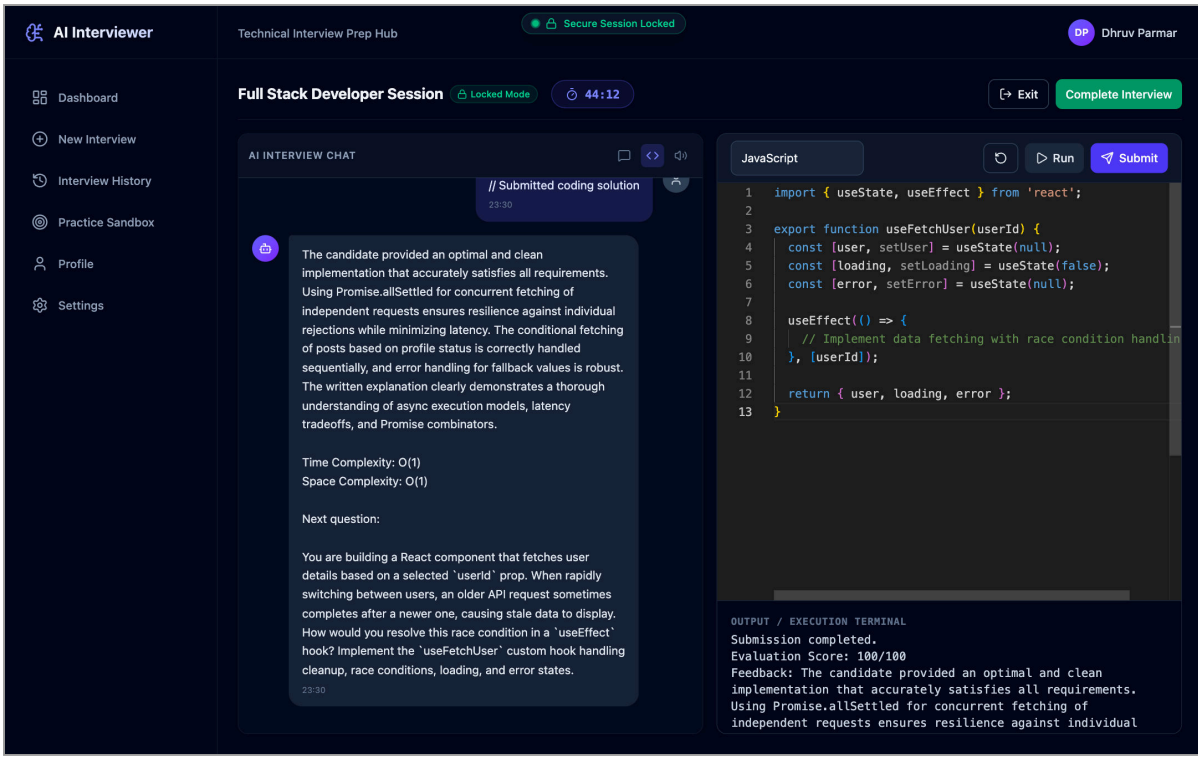



**Figure 5.9: AI Evaluation of a Coding Answer with Complexity Analysis**

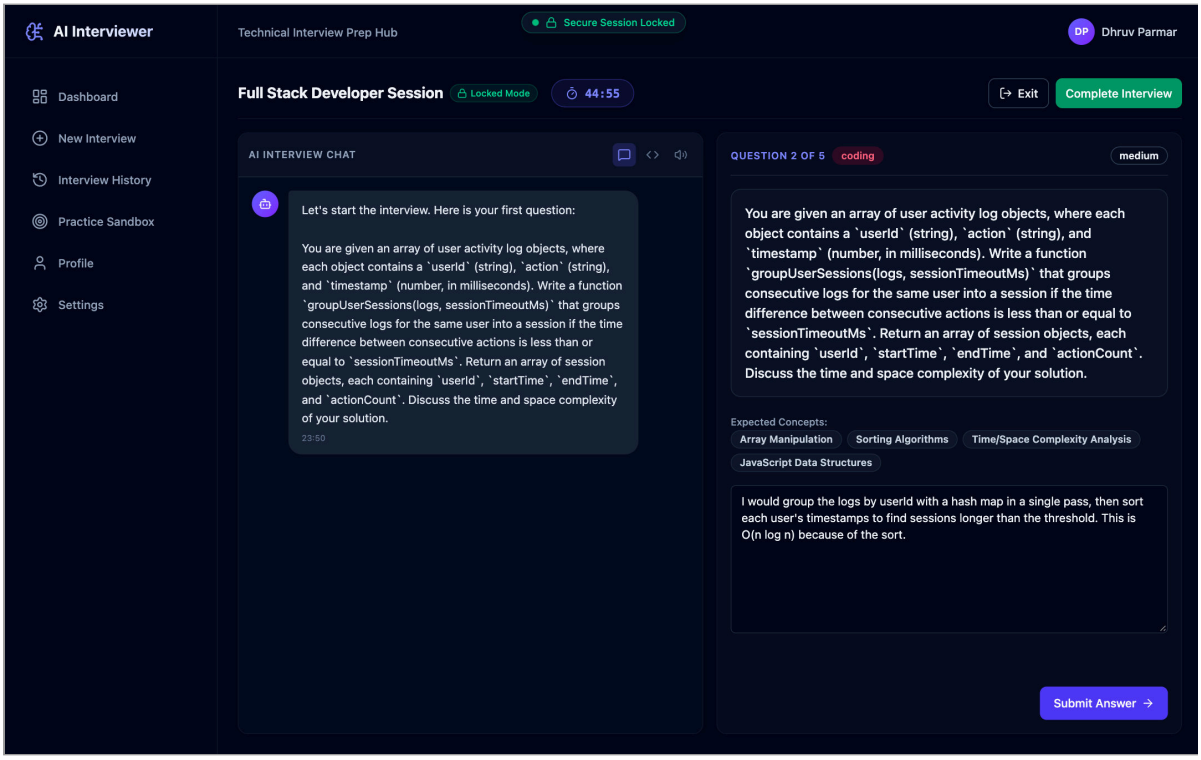



**Figure 5.10: Text Answer Mode with Question Panel**

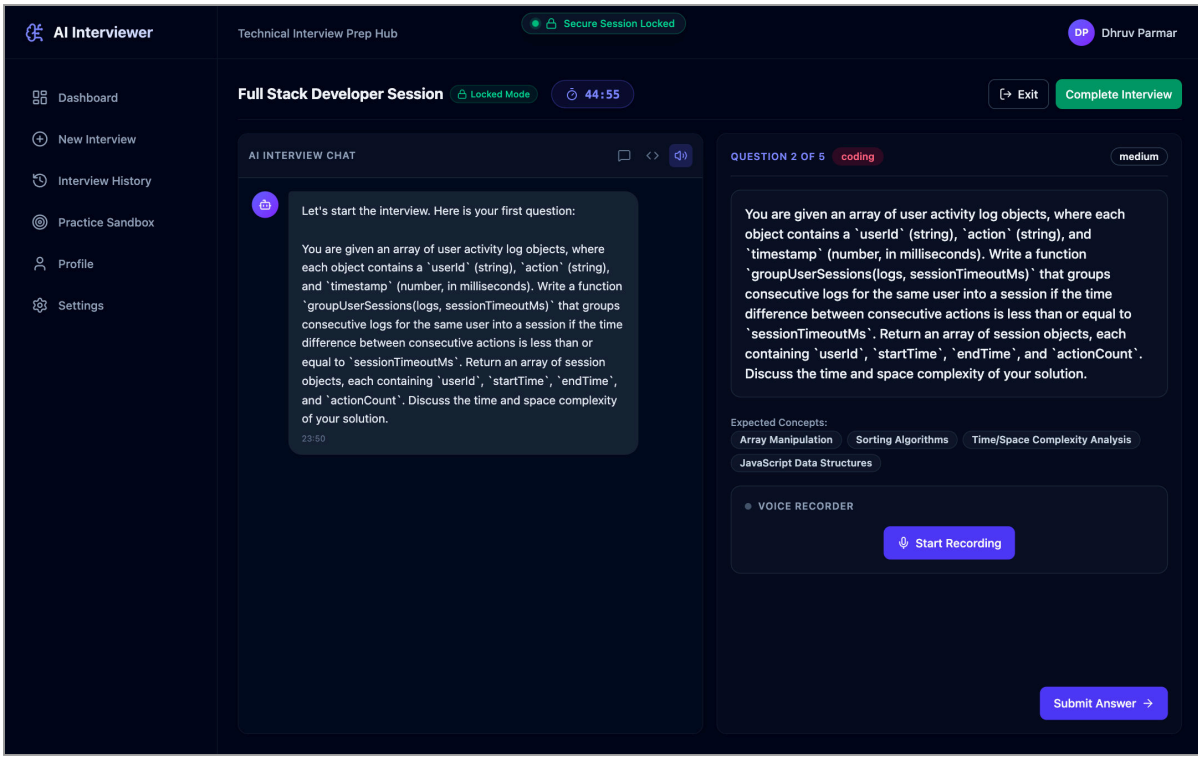



**Figure 5.11: Voice Answer Mode**

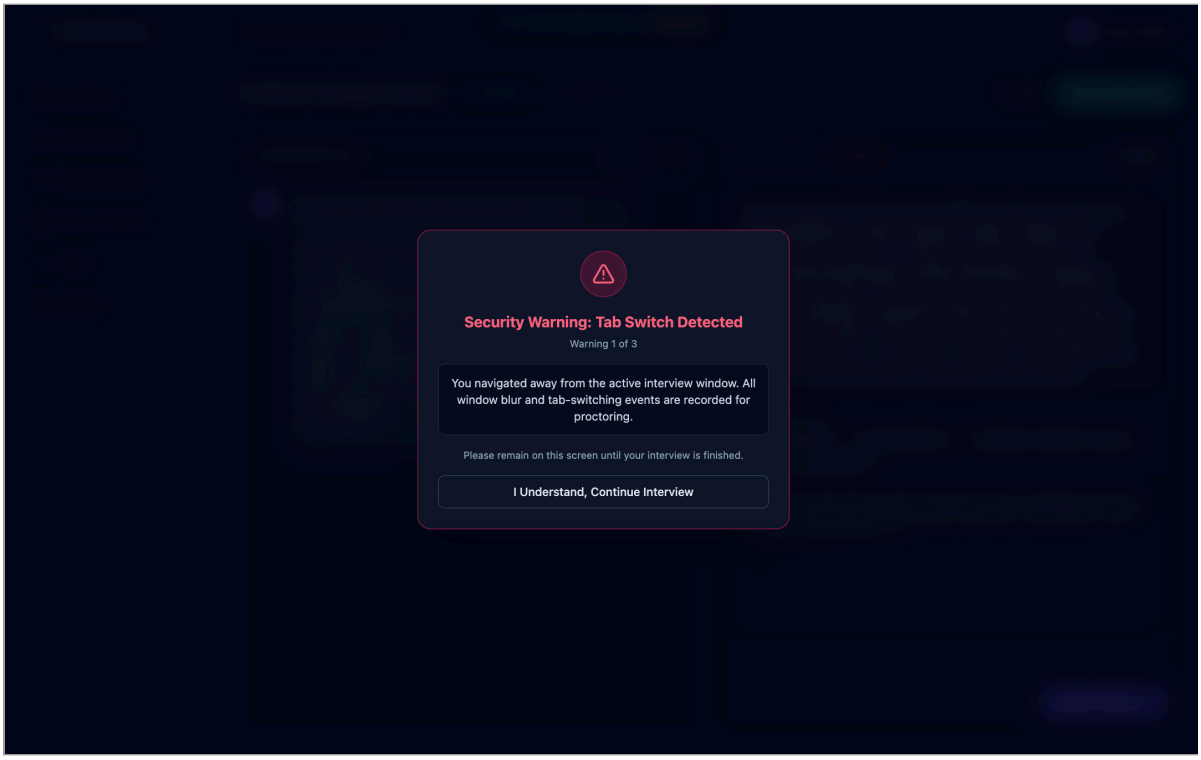



**Figure 5.12: Tab-Switch Proctoring Warning**

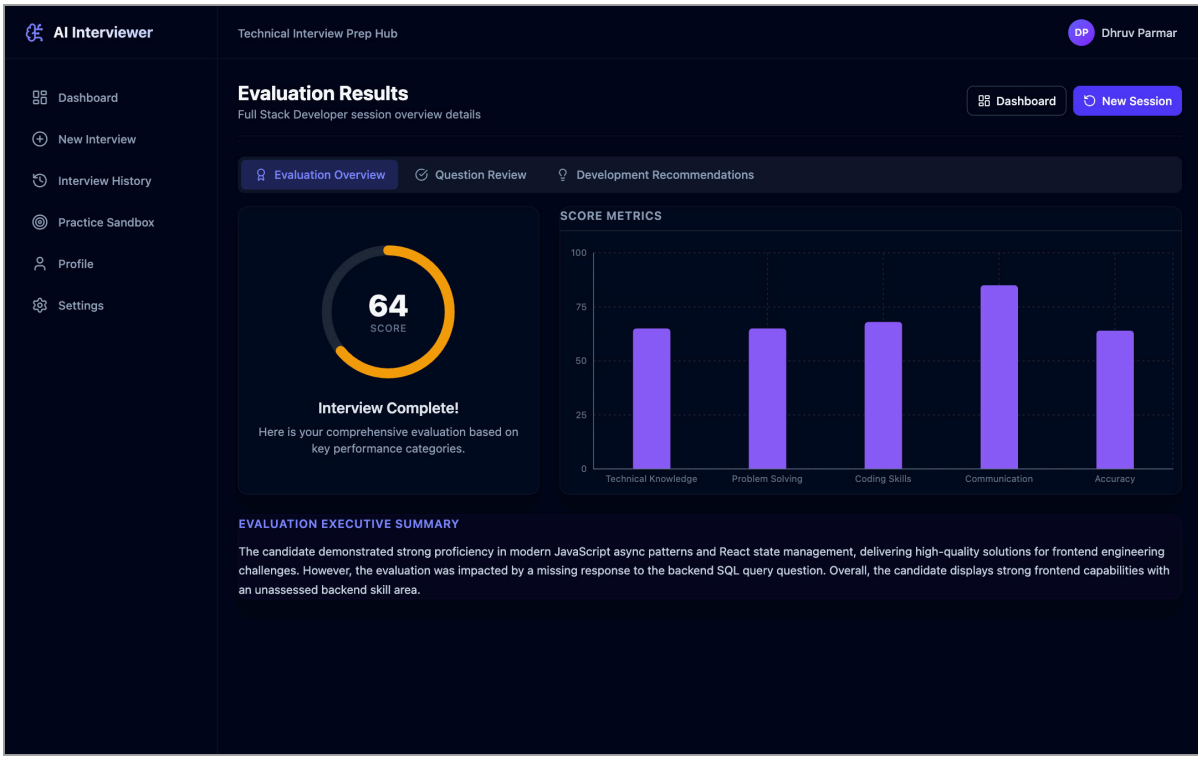



**Figure 5.13: Evaluation Results Overview**

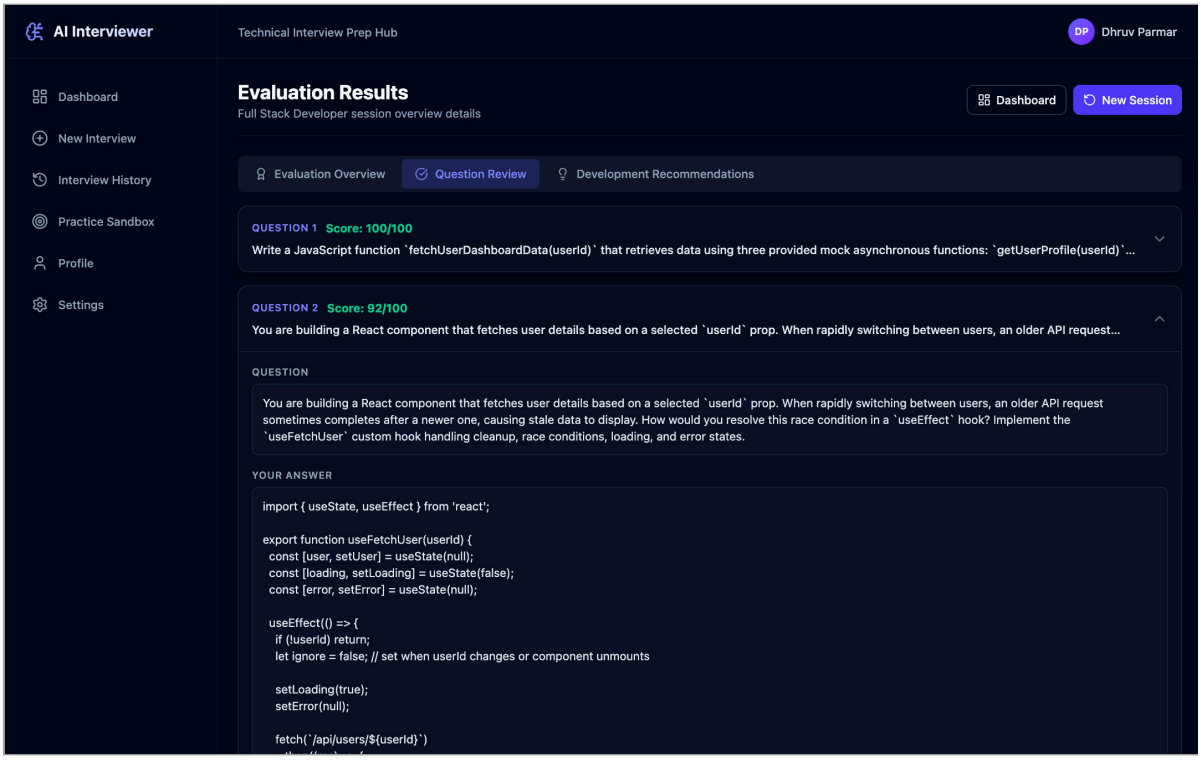



**Figure 5.14: Question Review with Answer and Score**

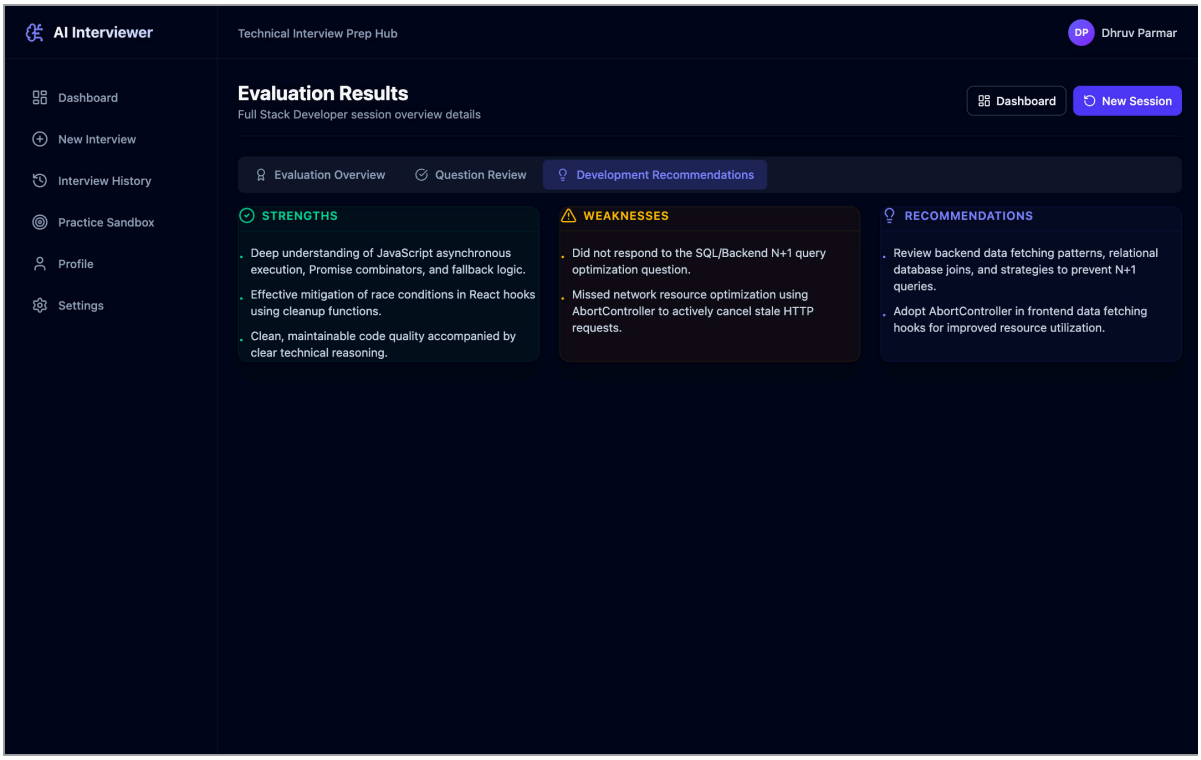



**Figure 5.15: Strengths, Weaknesses and Recommendations**

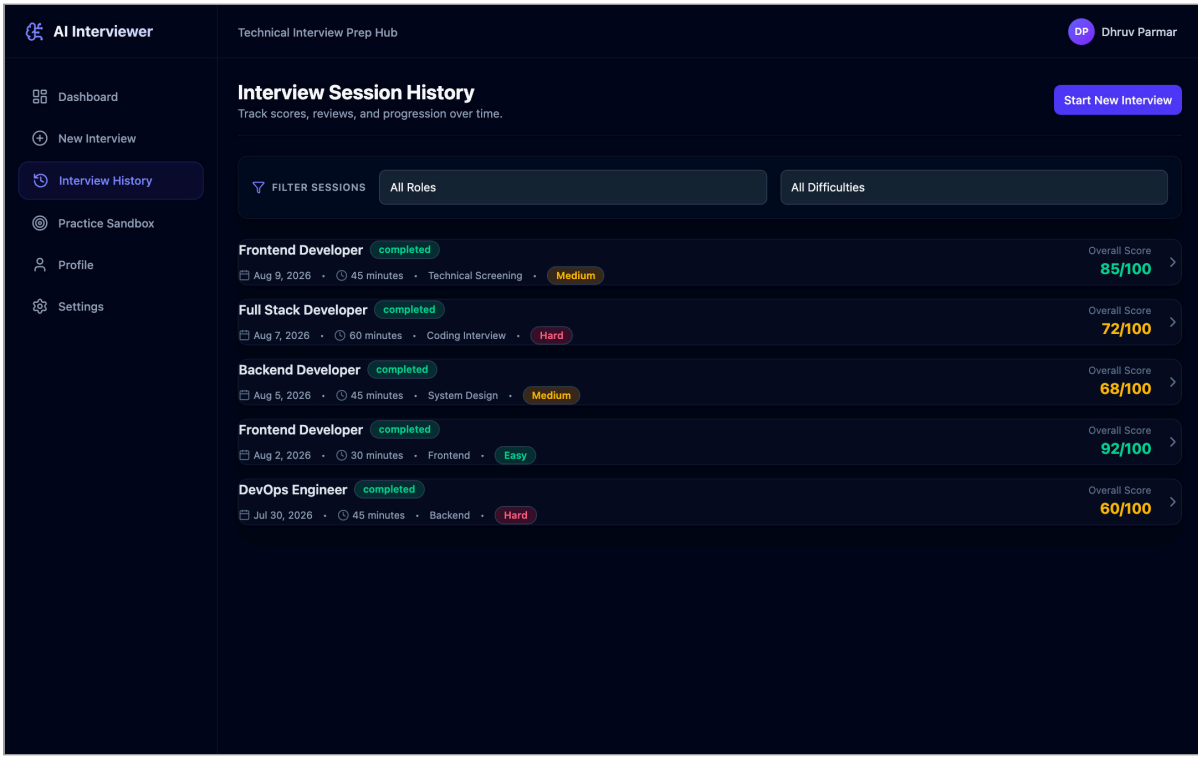



**Figure 5.16: Interview Session History (Sample Data for a New Account)**

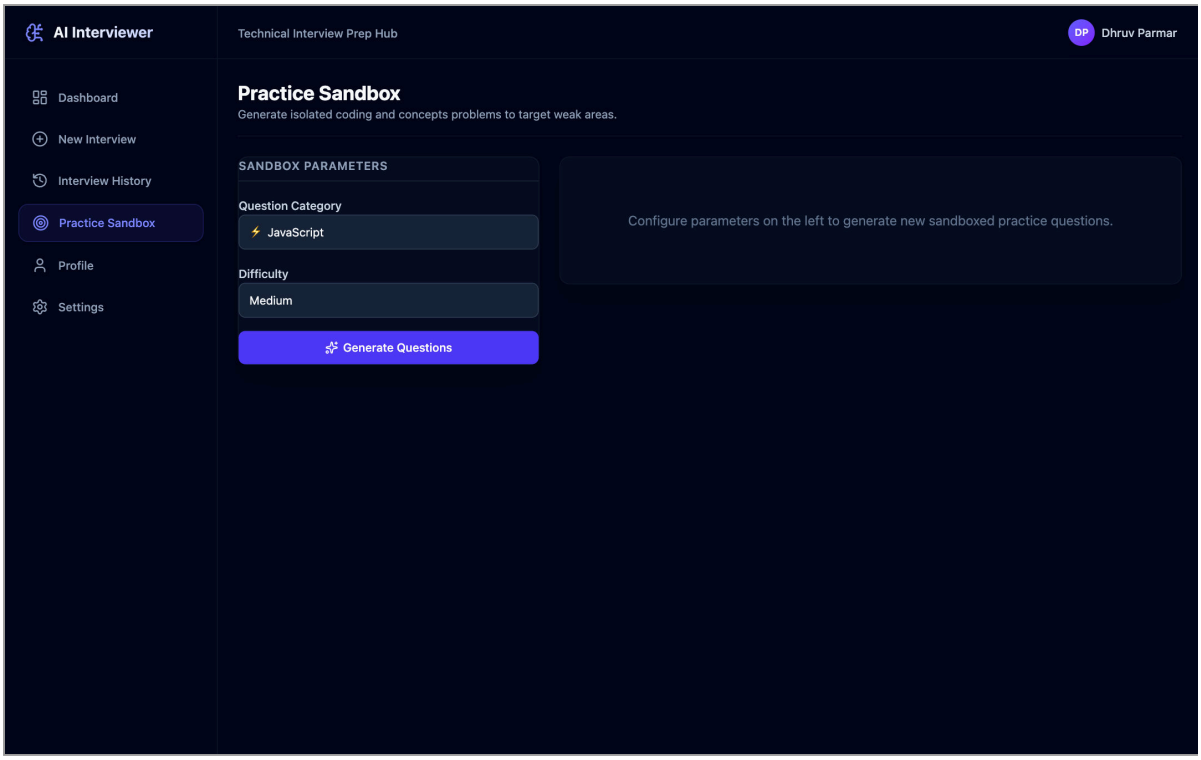



**Figure 5.17: Practice Sandbox**

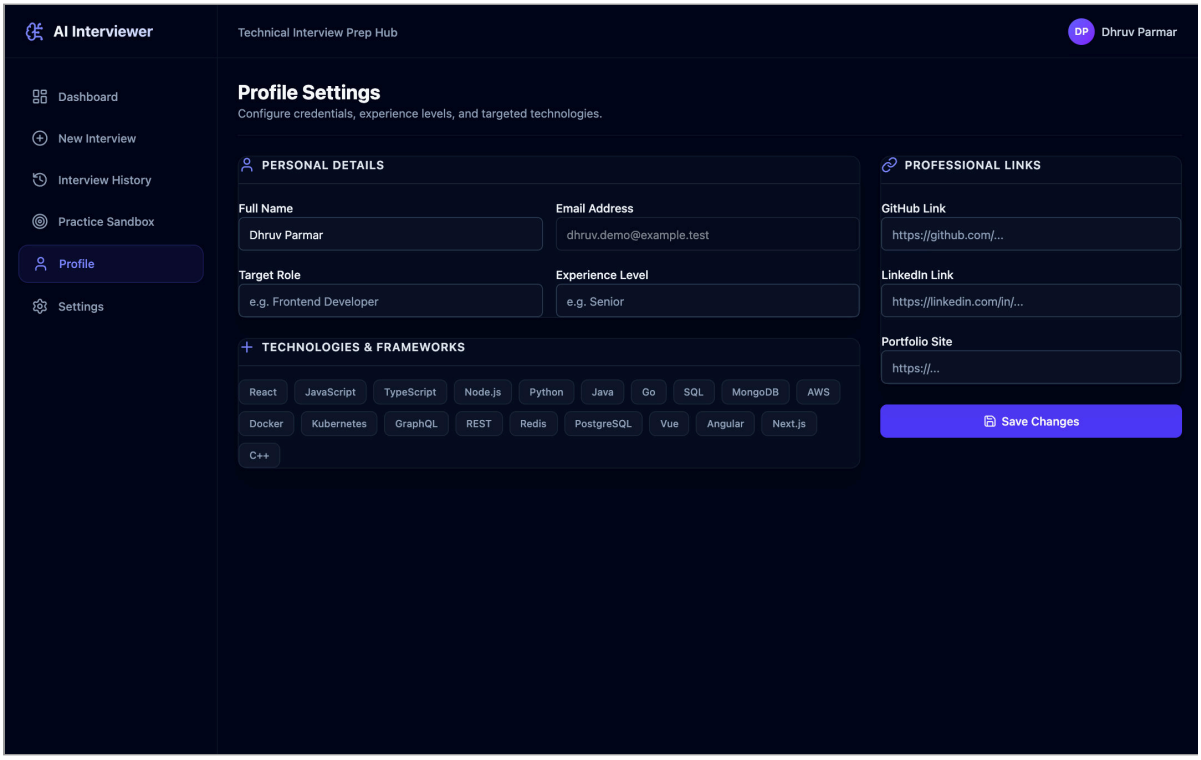



**Figure 5.18: Profile Settings**

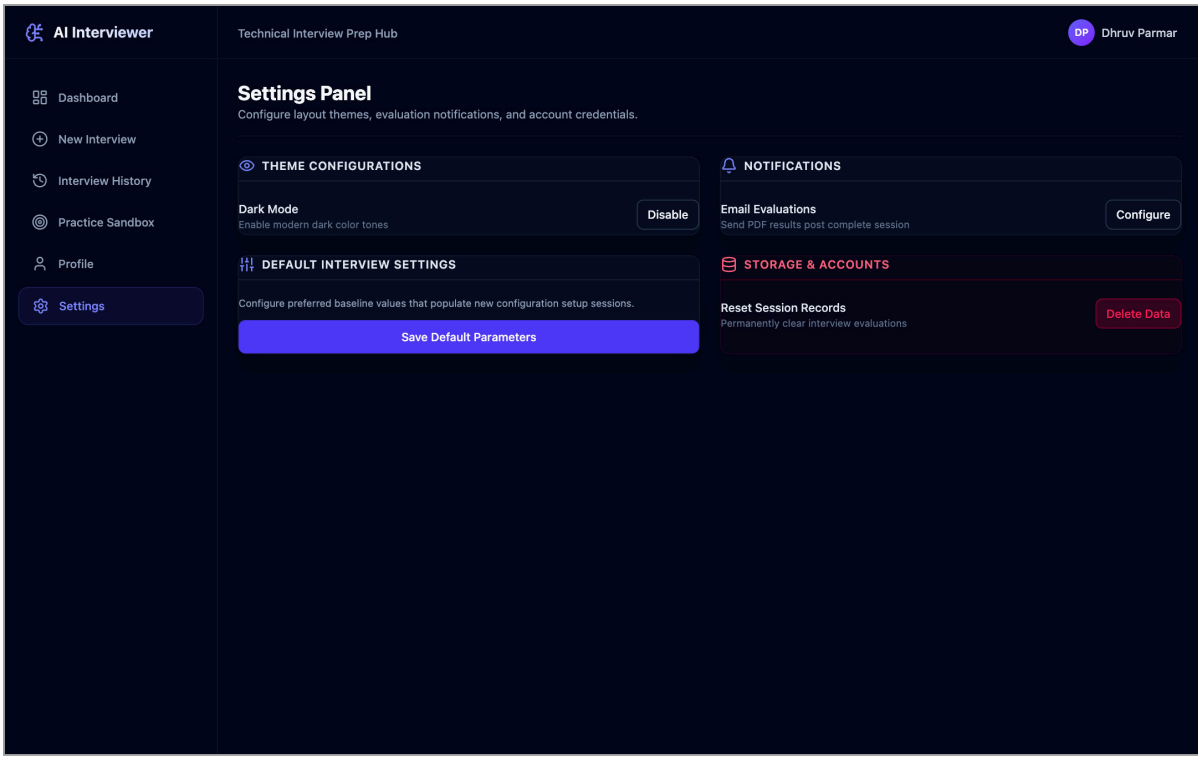



**Figure 5.19: Settings Panel**

# CHAPTER 6: CONCLUSION AND FUTURE SCOPE

## 6.1 CONCLUSION

The AI-Based Mock Interview Platform demonstrates that a large language model can conduct useful technical mock interviews when it is combined with an application that controls the interview structure. The platform lets a candidate configure an interview for a specific role, level and set of topics, answer AI-generated questions by text, voice or code in a proctored workspace, and receive immediate, specific feedback and a final report with skill-wise scores, strengths, weaknesses and recommendations.

The main design choices worked as intended. Generating one question at a time with topic rotation kept the interview focused on the selected areas. Requesting JSON responses and validating them with Zod schemas gave the rest of the application reliable data, and the fallback result calculation kept the results page available when the model could not be reached. The single follow-up rule bounded the length of a session, and the layered architecture with interfaces for the AI service, data store and code execution made it possible to switch to mock implementations for development. In testing with the live model, coding answers were evaluated with relevant, code-specific feedback, and the final report correctly reflected both strong answers and an unanswered question.

Testing also showed where the system needs further work: handling of AI service errors, linking dashboard statistics to stored results, real code execution and a few interface issues. Overall, the project achieves its aim of providing an accessible, structured and feedback-rich environment for technical interview practice, and it provides a solid base for the improvements listed below.

## 6.2 FUTURE SCOPE

1. **Reliable AI calls:** Add automatic retries with exponential back-off for temporary errors, clear messages for quota errors, and a fallback model.
2. **Real code execution:** Connect a sandboxed execution service such as Judge0, Piston or Docker containers and combine hidden test case results with the model’s code review.
3. **Accurate analytics:** Store the final score on each interview, record timestamps in every storage mode, and compute the dashboard and radar chart from real results.
4. **Robust session handling:** Make the start operation idempotent, restore the current question and chat history after a page reload, and return correct HTTP status codes for authentication errors.

5. **Speech-to-speech interviews:** Read questions aloud and accept continuous spoken answers for a fully voice-based interview.
6. **Multimodal analysis:** Optionally analyse webcam video and audio for confidence and communication cues, following work such as [3]–[5], with attention to privacy and fairness [6], [7].
7. **Resume and job-description based interviews:** Generate questions from the candidate's resume and a target job description.
8. **Placement cell portal:** Provide an administrative dashboard for Training and Placement cells to view progress, compare sessions and export reports.
9. **Calibration and evaluation:** Compare model scores with ratings from human interviewers on a larger set of sessions and refine the rubrics accordingly.

These enhancements would make the platform more reliable, more realistic and more useful for both individual students and institutions preparing candidates for technical interviews.

# REFERENCES

[1] F. L. Schmidt and J. E. Hunter, "The Validity and Utility of Selection Methods in Personnel Psychology: Practical and Theoretical Implications of 85 Years of Research Findings," *Psychological Bulletin*, vol. 124, no. 2, pp. 262–274, 1998. doi: 10.1037/0033-2909.124.2.262.

[2] M. A. Campion, D. K. Palmer, and J. E. Campion, "A Review of Structure in the Selection Interview," *Personnel Psychology*, vol. 50, no. 3, pp. 655–702, 1997. doi: 10.1111/j.1744-6570.1997.tb00709.x.

[3] M. E. Hoque, M. Courgeon, J.-C. Martin, B. Mutlu, and R. W. Picard, "MACH: My Automated Conversation coacH," in *Proc. ACM International Joint Conference on Pervasive and Ubiquitous Computing (UbiComp)*, pp. 697–706, 2013. doi: 10.1145/2493432.2493502.

[4] I. Naim, M. I. Tanveer, D. Gildea, and M. E. Hoque, "Automated Analysis and Prediction of Job Interview Performance," *IEEE Transactions on Affective Computing*, vol. 9, no. 2, pp. 191–204, 2018. doi: 10.1109/TAFFC.2016.2614299.

[5] L. Hemamou, G. Felhi, V. Vandenbussche, J.-C. Martin, and C. Clavel, "HireNet: A Hierarchical Attention Model for the Automatic Analysis of Asynchronous Video Job Interviews," in *Proc. AAAI Conference on Artificial Intelligence*, vol. 33, no. 1, pp. 573–581, 2019. doi: 10.1609/aaai.v33i01.3301573.

[6] L. Hickman, N. Bosch, V. Ng, R. Saef, L. Tay, and S. E. Woo, "Automated Video Interview Personality Assessments: Reliability, Validity, and Generalizability Investigations," *Journal of Applied Psychology*, vol. 107, no. 8, pp. 1323–1351, 2022. doi: 10.1037/apl0000695.

[7] M. Raghavan, S. Barocas, J. Kleinberg, and K. Levy, "Mitigating Bias in Algorithmic Hiring: Evaluating Claims and Practices," in *Proc. Conference on Fairness, Accountability, and Transparency (FAT*)*, pp. 469–481, 2020. doi: 10.1145/3351095.3372828.

[8] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention Is All You Need," in *Advances in Neural Information Processing Systems*, vol. 30, pp. 5998–6008, 2017.

[9] T. Brown et al., "Language Models are Few-Shot Learners," in *Advances in Neural Information Processing Systems*, vol. 33, pp. 1877–1901, 2020.

[10] L. Ouyang et al., "Training Language Models to Follow Instructions with Human Feedback," in *Advances in Neural Information Processing Systems*, vol. 35, pp. 27730–27744, 2022.

[11] OpenAI, "GPT-4 Technical Report," arXiv:2303.08774, 2023.

[12] Gemini Team, Google, “Gemini: A Family of Highly Capable Multimodal Models,” arXiv:2312.11805, 2023.

[13] J. Wei et al., “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” in *Advances in Neural Information Processing Systems*, vol. 35, pp. 24824–24837, 2022.

[14] P. Liu, W. Yuan, J. Fu, Z. Jiang, H. Hayashi, and G. Neubig, “Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods in Natural Language Processing,” *ACM Computing Surveys*, vol. 55, no. 9, Art. 195, pp. 1–35, 2023. doi: 10.1145/3560815.

[15] J. White et al., “A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT,” arXiv:2302.11382, 2023.

[16] M. Chen et al., “Evaluating Large Language Models Trained on Code,” arXiv:2107.03374, 2021.

[17] Y. Li et al., “Competition-Level Code Generation with AlphaCode,” *Science*, vol. 378, no. 6624, pp. 1092–1097, 2022. doi: 10.1126/science.abq1158.

[18] B. Rozière et al., “Code Llama: Open Foundation Models for Code,” arXiv:2308.12950, 2023.

[19] L. Zheng et al., “Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena,” in *Advances in Neural Information Processing Systems*, vol. 36 (Datasets and Benchmarks Track), 2023.

[20] Z. Ji et al., “Survey of Hallucination in Natural Language Generation,” *ACM Computing Surveys*, vol. 55, no. 12, Art. 248, pp. 1–38, 2023. doi: 10.1145/3571730.

[21] E. Kasneci et al., “ChatGPT for Good? On Opportunities and Challenges of Large Language Models for Education,” *Learning and Individual Differences*, vol. 103, Art. 102274, 2023. doi: 10.1016/j.lindif.2023.102274.

[22] G. Kurdi, J. Leo, B. Parsia, U. Sattler, and S. Al-Emari, “A Systematic Review of Automatic Question Generation for Educational Purposes,” *International Journal of Artificial Intelligence in Education*, vol. 30, pp. 121–204, 2020. doi: 10.1007/s40593-019-00186-y.

[23] R. T. Fielding, “Architectural Styles and the Design of Network-based Software Architectures,” Ph.D. dissertation, University of California, Irvine, 2000.

[24] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” RFC 7519, Internet Engineering Task Force, May 2015. doi: 10.17487/RFC7519.

[25] N. Provos and D. Mazières, “A Future-Adaptable Password Scheme,” in *Proc. USENIX Annual Technical Conference, FREENIX Track*, 1999.

[26] OWASP Foundation, “OWASP Top 10: 2021.” Available: https://owasp.org/Top10/

[27] Google, “Gemini API Documentation.” Available: https://ai.google.dev/gemini-api/docs

[28] Meta Open Source, “React Documentation.” Available: https://react.dev/

[29] OpenJS Foundation, “Node.js Documentation” and “Express Documentation.” Available: https://nodejs.org/docs/ and https://expressjs.com/

[30] Prisma Data, “Prisma ORM Documentation.” Available: https://www.prisma.io/docs

[31] Zod, “Zod Documentation.” Available: https://zod.dev/

[32] Microsoft, “Monaco Editor.” Available: https://microsoft.github.io/monaco-editor/

[33] MDN Web Docs, “Web Speech API.” Available: https://developer.mozilla.org/en-US/docs/Web/API/Web_Speech_API

[34] MDN Web Docs, “Fullscreen API” and “Page Visibility API.” Available: https://developer.mozilla.org/en-US/docs/Web/API/Fullscreen_API and https://developer.mozilla.org/en-US/docs/Web/API/Page_Visibility_API

[35] Vite, “Vite Documentation.” Available: https://vite.dev/guide/

[36] Tailwind Labs, “Tailwind CSS Documentation.” Available: https://tailwindcss.com/docs

[37] Recharts, “Recharts Documentation.” Available: https://recharts.org/

[38] Poimandres, “Zustand Documentation.” Available: https://zustand.docs.pmnd.rs/